# Final project notebook: earnings calls and CRSP daily returns

```r
library(dplyr)
library(tidyr)
library(readr)
library(tibble)
library(stringr)
library(lubridate)
library(zoo)
library(slider)
library(ggplot2)
library(reshape2)
library(lightgbm)
library(xgboost)
```

This notebook builds a **daily security–date panel**: CRSP daily fields merged with **prior-quarter** transcript-based metrics, then models **next-day** `dlyretx` (with `next_day_return` from `dlyret` kept for diagnostics) using **LightGBM**, **XGBoost**, optional **Random Forest**, and **walk-forward time-series CV** for hyperparameter tuning.

# How to run

Run chunks **top to bottom** in order. The first code chunk loads R packages; paths point to `Data/Raw Data/` under this project (adjust if your layout differs). Later sections mirror the Python notebook's **GPU** notes where relevant; this R port runs **LightGBM / XGBoost on CPU** by default.

# Main outputs

- Cleaned transcript universe aligned with CRSP tickers
- `merged_daily_df` / `model_df` with transcript **missingness flags**, **z-scored** transcript metrics and dynamics (train-fitted scaler), **`dlyret` / `dlyretx` lags**, rolling stats, **OHLC microstructure**, optional **cross-sectional** features, **`FE_STAGE`** ablation switch, and encoded categoricals ( `ticker_enc` , `siccd_enc` , `naics_enc` )
- Time-based train / validation / test split and model comparison metrics

# 1. Data loading

- Load CRSP daily ( `crsp_v2_2014_2024.csv` ) and the wide transcript metrics file.
- Use `glimpse()` to confirm dtypes, row counts, and column names before cleaning.

```
# --- Load raw CRSP and transcript CSVs; print schema ---
crsp_df <- read_csv(
  "H:/My Drive/Project/FA Project/Data/Raw Data/crsp_v2_2014_2024.csv",
  show_col_types = FALSE
)
transcripts_df <- read_csv(
  "H:/My Drive/Project/FA Project/Data/Raw Data/Transcripts Machine Readable Data.csv",
  show_col_types = FALSE
)
glimpse(crsp_df)
```

```
## Rows: 495,987
## Columns: 27
## $ permno       <dbl> 10104, 10104, 10104, 10104, 10104, 10104, 10104, 10104, 1…
## $ permco       <dbl> 8045, 8045, 8045, 8045, 8045, 8045, 8045, 8045, 8045, 804…
## $ ticker       <chr> "ORCL", "ORCL", "ORCL", "ORCL", "ORCL", "ORCL", "ORCL", "…
## $ cusip        <chr> "68389X10", "68389X10", "68389X10", "68389X10", "68389X10…
## $ issuernm     <chr> "ORACLE CORP", "ORACLE CORP", "ORACLE CORP", "ORACLE CORP…
## $ siccd        <dbl> 7372, 7372, 7372, 7372, 7372, 7372, 7372, 7372, 7372, 737…
## $ naics        <dbl> 511210, 511210, 511210, 511210, 511210, 511210, 511210, 5…
## $ dlycaldt     <date> 2014-01-02, 2014-01-03, 2014-01-06, 2014-01-07, 2014-01-…
## $ dlyclose     <dbl> 37.84, 37.62, 37.47, 37.85, 37.72, 37.65, 38.11, 37.75, 3…
## $ dlyopen      <dbl> 37.78, 37.65, 37.64, 37.66, 37.79, 37.85, 37.75, 37.95, 3…
## $ dlyhigh      <dbl> 38.0300, 37.8600, 37.8000, 37.9300, 37.9100, 37.8500, 38.…
## $ dlylow       <dbl> 37.550, 37.560, 37.415, 37.500, 37.560, 37.460, 37.590, 3…
## $ dlyprc       <dbl> 37.84, 37.62, 37.47, 37.85, 37.72, 37.65, 38.11, 37.75, 3…
## $ dlyret       <dbl> -0.010978, -0.002643, -0.003987, 0.010141, -0.003435, -0.…
## $ dlyretx      <dbl> -0.010978, -0.005814, -0.003987, 0.010141, -0.003435, -0.…
## $ dlyvol       <dbl> 18163700, 11693900, 15330000, 16793300, 16111600, 1362350…
## $ shrout       <dbl> 4497409, 4497409, 4497409, 4497409, 4497409, 4497409, 449…
## $ dlycap       <dbl> 170181957, 169192527, 168517915, 170226931, 169642267, 16…
## $ dlycumfacpr  <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, …
## $ dlycumfacshr <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, …
## $ sprtrn       <dbl> -0.008862, -0.000333, -0.002512, 0.006082, -0.000212, 0.0…
## $ vwretd       <dbl> -0.008757, 0.000491, -0.003340, 0.006090, 0.000155, 0.000…
## $ ewretd       <dbl> -0.004051, 0.004096, -0.001676, 0.006892, 0.000835, 0.000…
## $ securitytype <chr> "EQTY", "EQTY", "EQTY", "EQTY", "EQTY", "EQTY", "EQTY", "…
## $ sharetype    <chr> "NS", "NS", "NS", "NS", "NS", "NS", "NS", "NS", "NS", "NS…
## $ usincflg     <chr> "Y", "Y", "Y", "Y", "Y", "Y", "Y", "Y", "Y", "Y", "Y", "Y…
## $ shradrflg    <chr> "N", "N", "N", "N", "N", "N", "N", "N", "N", "N", "N", "N…
```

```
glimpse(transcripts_df)
```

```
## Rows: 772
## Columns: 48
## $ Symbol         <chr> "T", "GOOGL", "GOOG", "META", "NFLX", "NWSA", "OMC", "V…
## $ `Company Name` <chr> "AT&T", "Alphabet Inc. (Class A)", "Alphabet Inc. (Clas…
## $ Metrics        <chr> "Net Positivity", "Net Positivity", "Net Positivity", "…
## $ CQ12014        <chr> "1.54", "1.42", "#INVALID COMPANY ID", "1.64", "0.98", …
## $ CQ22014        <chr> "1.86", "1.37", "#INVALID COMPANY ID", "1.69", "1.22", …
## $ CQ32014        <chr> "1.85", "1.25", "#INVALID COMPANY ID", "1.78", "1.38", …
## $ CQ42014        <chr> "1.36", "1.48", "#INVALID COMPANY ID", "1.43", "0.50", …
## $ CQ12015        <chr> "1.14", "0.97", "#INVALID COMPANY ID", "1.74", "0.98", …
## $ CQ22015        <chr> "1.54", "1.82", "#INVALID COMPANY ID", "1.88", "0.99", …
## $ CQ32015        <chr> "2.01", "1.68", "#INVALID COMPANY ID", "1.48", "1.19", …
## $ CQ42015        <chr> "1.81", "1.56", "#INVALID COMPANY ID", "1.54", "1.16", …
## $ CQ12016        <chr> "1.50", "1.74", "#INVALID COMPANY ID", "1.65", "1.10", …
## $ CQ22016        <chr> "2.24", "2.03", "#INVALID COMPANY ID", "1.21", "1.22", …
## $ CQ32016        <chr> "1.74", "1.90", "#INVALID COMPANY ID", "1.53", "0.84", …
## $ CQ42016        <chr> "1.81", "1.95", "#INVALID COMPANY ID", "1.46", "1.05", …
## $ CQ12017        <chr> "1.61", "1.90", "#INVALID COMPANY ID", "1.26", "1.69", …
## $ CQ22017        <chr> "0.83", "2.24", "#INVALID COMPANY ID", "0.95", "1.60", …
## $ CQ32017        <chr> "1.42", "1.80", "#INVALID COMPANY ID", "1.44", "2.00", …
## $ CQ42017        <chr> "1.46", "1.77", "#INVALID COMPANY ID", "0.99", "1.53", …
## $ CQ12018        <chr> "1.78", "1.81", "#INVALID COMPANY ID", "1.32", "1.31", …
## $ CQ22018        <chr> "1.54", "2.06", "#INVALID COMPANY ID", "1.25", "1.38", …
## $ CQ32018        <chr> "1.66", "1.96", "#INVALID COMPANY ID", "1.36", "1.80", …
## $ CQ42018        <chr> "1.40", "1.72", "#INVALID COMPANY ID", "1.42", "1.15", …
## $ CQ12019        <chr> "1.35", "1.95", "#INVALID COMPANY ID", "1.74", "1.66", …
## $ CQ22019        <chr> "1.42", "1.66", "#INVALID COMPANY ID", "0.93", "1.86", …
## $ CQ32019        <chr> "1.32", "1.61", "#INVALID COMPANY ID", "1.14", "1.74", …
## $ CQ42019        <chr> "1.52", "1.78", "#INVALID COMPANY ID", "0.95", "1.66", …
## $ CQ12020        <chr> "1.57", "1.70", "#INVALID COMPANY ID", "1.04", "1.36", …
## $ CQ22020        <chr> "0.22", "0.72", "#INVALID COMPANY ID", "0.00", "1.14", …
## $ CQ32020        <chr> "0.98", "1.58", "#INVALID COMPANY ID", "0.69", "2.00", …
## $ CQ42020        <chr> "1.39", "1.87", "#INVALID COMPANY ID", "0.63", "0.97", …
## $ CQ12021        <chr> "1.11", "1.63", "#INVALID COMPANY ID", "0.80", "1.34", …
## $ CQ22021        <chr> "1.45", "1.89", "#INVALID COMPANY ID", "1.50", "1.52", …
## $ CQ32021        <chr> "1.32", "1.70", "#INVALID COMPANY ID", "1.61", "1.35", …
## $ CQ42021        <chr> "1.26", "1.38", "#INVALID COMPANY ID", "1.13", "1.96", …
## $ CQ12022        <chr> "1.06", "1.85", "#INVALID COMPANY ID", "0.68", "1.29", …
## $ CQ22022        <chr> "1.31", "1.63", "#INVALID COMPANY ID", "1.10", "1.51", …
## $ CQ32022        <chr> "1.18", "1.56", "#INVALID COMPANY ID", "1.15", "1.55", …
## $ CQ42022        <chr> "1.60", "1.35", "#INVALID COMPANY ID", "1.09", "1.49", …
## $ CQ12023        <chr> "1.22", "2.11", "#INVALID COMPANY ID", "1.35", "1.79", …
## $ CQ22023        <chr> "1.28", "2.25", "#INVALID COMPANY ID", "1.90", "1.95", …
## $ CQ32023        <chr> "1.35", "2.50", "#INVALID COMPANY ID", "1.25", "1.50", …
## $ CQ42023        <chr> "1.43", "2.34", "#INVALID COMPANY ID", "1.74", "1.61", …
## $ CQ12024        <chr> "1.64", "2.50", "#INVALID COMPANY ID", "2.00", "1.78", …
## $ CQ22024        <chr> "1.14", "2.52", "#INVALID COMPANY ID", "2.10", "1.96", …
## $ CQ32024        <chr> "1.62", "2.44", "#INVALID COMPANY ID", "2.40", "2.32", …
## $ CQ42024        <chr> "1.35", "2.22", "#INVALID COMPANY ID", "2.02", "2.18", …
## $ ...48          <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,…
```

# 2. Ticker coverage (transcripts vs CRSP)

Identify symbols that appear in **transcripts** but not in **CRSP**, and decide how to restrict the transcript universe so merges are reliable.

## Tickers in transcripts missing from CRSP

List symbols present in the transcript file but **not** in CRSP `ticker`, before we restrict the transcript universe.

```
# --- Compare unique tickers: transcripts vs CRSP ---
crsp_tickers <- unique(crsp_df$ticker)
transcripts_tickers <- unique(transcripts_df$Symbol)
cat(length(unique(transcripts_df$Symbol)), "---", length(unique(crsp_df$ticker)), "\n")
```

```
## 193 --- 185
```

```
for (ticker in transcripts_tickers) {
  if (!ticker %in% crsp_tickers) {
    co_name <- unique(transcripts_df$`Company Name`[transcripts_df$Symbol == ticker])
    cat(ticker, paste0("['", paste(co_name, collapse = "', '"), "']"), "\n")
  }
}
```

```
## BF.B ['Brown-Forman']
## SLB ['Schlumberger']
## AON ['Aon plc']
## BRK.B ['Berkshire Hathaway']
## IVZ ['Invesco']
## ETN ['Eaton Corporation']
## ACN ['Accenture']
## EQR ['Equity Residential']
```

## Tickers dropped from transcripts

The previous chunk lists companies present in **transcripts** but not in **CRSP** (e.g. alternate share classes). Those tickers are removed from the transcript table so later merges use a consistent symbol set.

# 3. Exploratory missingness

Quick **null counts** on the raw CRSP and transcript tables after the initial universe decision.

```
# --- CRSP: null counts per column ---
colSums(is.na(crsp_df))
```

```
##       permno       permco       ticker        cusip     issuernm        siccd
##            0            0            0            0            0            0
##        naics     dlycaldt     dlyclose      dlyopen      dlyhigh       dlylow
##            0            0          277          277          277          277
##       dlyprc       dlyret      dlyretx       dlyvol       shrout       dlycap
##            1            3            3            1            0            1
##  dlycumfacpr dlycumfacshr       sprtrn       vwretd       ewretd securitytype
##            0            0            0            0            0            0
##    sharetype     usincflg    shradrflg
##            0            0            0
```

```
# --- Transcripts: null counts per column ---
colSums(is.na(transcripts_df))
```

```
##       Symbol Company Name      Metrics      CQ12014      CQ22014      CQ32014
##            0            0            0           20           24           32
##      CQ42014      CQ12015      CQ22015      CQ32015      CQ42015      CQ12016
##           28           20           40           32           28           32
##      CQ22016      CQ32016      CQ42016      CQ12017      CQ22017      CQ32017
##           32           36           44           24           24           24
##      CQ42017      CQ12018      CQ22018      CQ32018      CQ42018      CQ12019
##           24           28           28           28           28           24
##      CQ22019      CQ32019      CQ42019      CQ12020      CQ22020      CQ32020
##           28           32           24           33           32           20
##      CQ42020      CQ12021      CQ22021      CQ32021      CQ42021      CQ12022
##           28           20           28           20           24           28
##      CQ22022      CQ32022      CQ42022      CQ12023      CQ22023      CQ32023
##           28           24           20           20           20           20
##      CQ42023      CQ12024      CQ22024      CQ32024      CQ42024        ...48
##           20           16           20           24           16          772
```

# 4. Transcript filtering and quarterly missingness

Keep transcript rows in the CRSP ticker set, analyze **quarterly (CQ*)** missing values by ticker, optionally drop the worst tickers, and summarize missing rates.

```
# 1) Filter transcript rows for symbols absent from CRSP
crsp_tickers_set <- unique(na.omit(crsp_df$ticker))
transcript_tickers_set <- unique(na.omit(transcripts_df$Symbol))
missing_symbols <- sort(setdiff(transcript_tickers_set, crsp_tickers_set))

cat("Symbols in transcripts but not in CRSP:", paste0("['", paste(missing_symbols, collapse =
"', '"), "']"), "\n")
```

```
## Symbols in transcripts but not in CRSP: ['ACN', 'AON', 'BF.B', 'BRK.B', 'EQR', 'ETN', 'IVZ',
'SLB']
```

```
cat("Count:", length(missing_symbols), "\n")
```

```
## Count: 8
```

```
transcripts_filtered_df <- transcripts_df %>%
  filter(!Symbol %in% missing_symbols)

# Drop unnamed columns if present
unnamed_cols <- grep("^Unnamed", colnames(transcripts_filtered_df), value = TRUE)
if (length(unnamed_cols) > 0) {
  transcripts_filtered_df <- transcripts_filtered_df %>% select(-all_of(unnamed_cols))
}

cat("Unique symbols before:", length(unique(transcripts_df$Symbol)), "\n")
```

```
## Unique symbols before: 193
```

```
cat("Unique symbols after:", length(unique(transcripts_filtered_df$Symbol)), "\n")
```

```
## Unique symbols after: 185
```

```
cat("Rows before:", nrow(transcripts_df), "| Rows after:", nrow(transcripts_filtered_df), "\n")
```

```
## Rows before: 772 | Rows after: 740
```

```
# 2) Identify which tickers have missing values in OHLC columns
ohlc_cols <- c("dlyclose", "dlyhigh", "dlyopen", "dlylow")
missing_ohlc_mask <- rowSums(is.na(crsp_df[, ohlc_cols])) > 0

missing_ohlc_ticker_counts <- sort(table(crsp_df$ticker[missing_ohlc_mask]), decreasing = TRUE)
cat("Ticker-level missing OHLC counts:\n")
```

```
## Ticker-level missing OHLC counts:
```

```
print(missing_ohlc_ticker_counts)
```

```
##
##  MKC BIIB
##  275    2
```

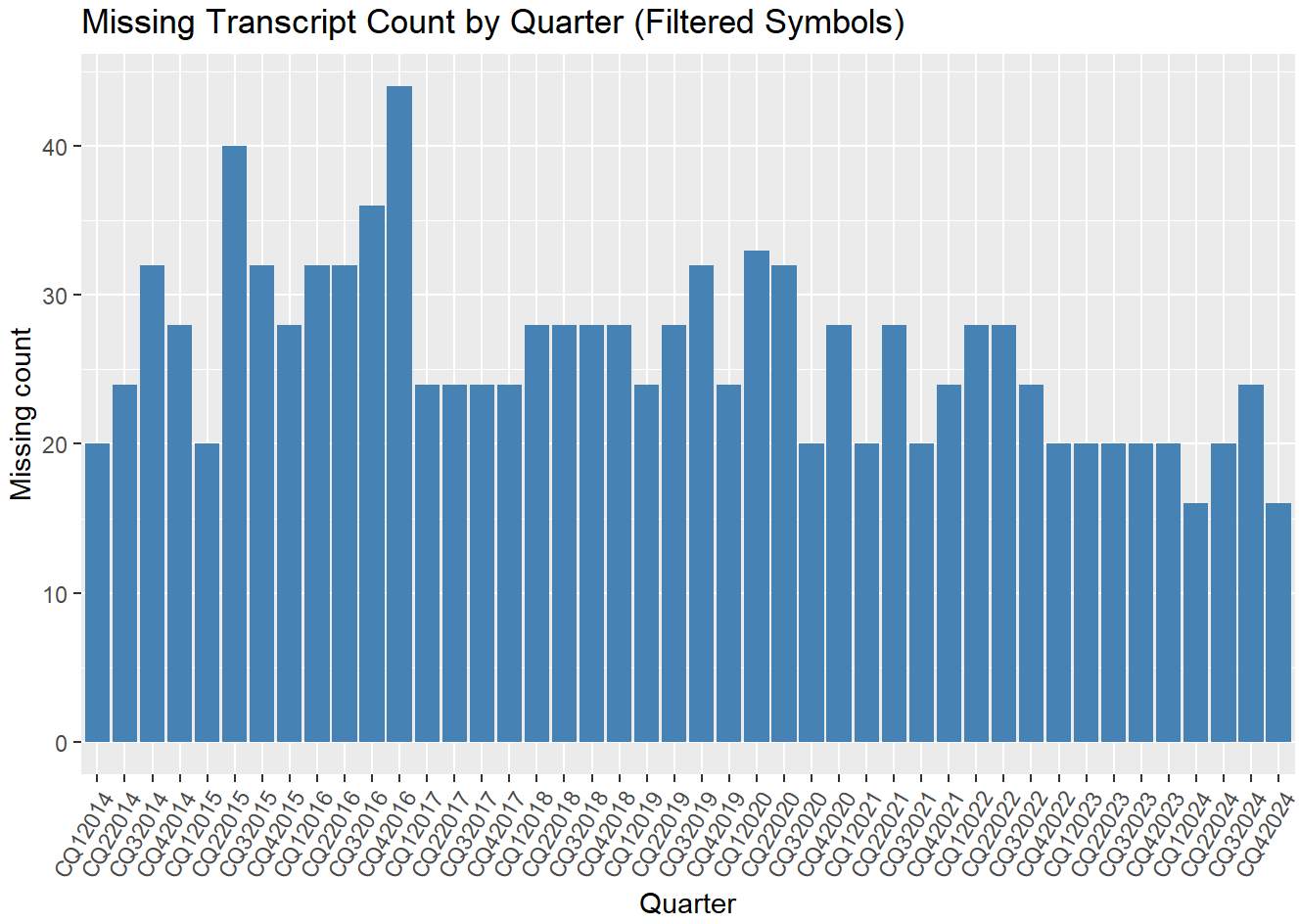
```
missing_ohlc_details <- crsp_df[missing_ohlc_mask, c("permno", "ticker", "dlycaldt", ohlc_cols)]
%>%
  arrange(ticker, dlycaldt)

cat("\nSample missing OHLC rows:\n")
```

```
##
## Sample missing OHLC rows:
```

```
print(head(missing_ohlc_details, 20))
```

```
## # A tibble: 20 × 7
##    permno ticker dlycaldt   dlyclose dlyhigh dlyopen dlylow
##     <dbl> <chr>  <date>        <dbl>   <dbl>   <dbl>  <dbl>
##  1  76841 BIIB   2020-11-06       NA      NA      NA     NA
##  2  76841 BIIB   2023-06-09       NA      NA      NA     NA
##  3  89155 MKC    2014-01-10       NA      NA      NA     NA
##  4  89155 MKC    2014-01-24       NA      NA      NA     NA
##  5  89155 MKC    2014-01-27       NA      NA      NA     NA
##  6  89155 MKC    2014-02-05       NA      NA      NA     NA
##  7  89155 MKC    2014-02-13       NA      NA      NA     NA
##  8  89155 MKC    2014-02-20       NA      NA      NA     NA
##  9  89155 MKC    2014-02-25       NA      NA      NA     NA
## 10  89155 MKC    2014-02-28       NA      NA      NA     NA
## 11  89155 MKC    2014-03-04       NA      NA      NA     NA
## 12  89155 MKC    2014-03-18       NA      NA      NA     NA
## 13  89155 MKC    2014-04-01       NA      NA      NA     NA
## 14  89155 MKC    2014-04-16       NA      NA      NA     NA
## 15  89155 MKC    2014-04-23       NA      NA      NA     NA
## 16  89155 MKC    2014-04-25       NA      NA      NA     NA
## 17  89155 MKC    2014-04-30       NA      NA      NA     NA
## 18  89155 MKC    2014-05-07       NA      NA      NA     NA
## 19  89155 MKC    2014-05-08       NA      NA      NA     NA
## 20  89155 MKC    2014-05-12       NA      NA      NA     NA
```

```
# 3) Plot missing count by quarter in transcripts (filtered universe)
quarter_cols <- grep("^CQ[1-4]\\d{4}$", colnames(transcripts_filtered_df), value = TRUE)

quarter_sort_key <- function(col_name) {
  q <- as.integer(substr(col_name, 3, 3))
  y <- as.integer(substr(col_name, 4, nchar(col_name)))
  y * 10 + q
}
quarter_cols <- quarter_cols[order(sapply(quarter_cols, quarter_sort_key))]
missing_per_quarter <- colSums(is.na(transcripts_filtered_df[, quarter_cols]))

missing_per_quarter_df <- tibble(
  quarter = factor(quarter_cols, levels = quarter_cols),
  missing_count = as.integer(missing_per_quarter)
)

ggplot(missing_per_quarter_df, aes(x = quarter, y = missing_count)) +
  geom_col(fill = "steelblue") +
  labs(title = "Missing Transcript Count by Quarter (Filtered Symbols)",
       x = "Quarter", y = "Missing count") +
  theme(axis.text.x = element_text(angle = 60, hjust = 1))
```



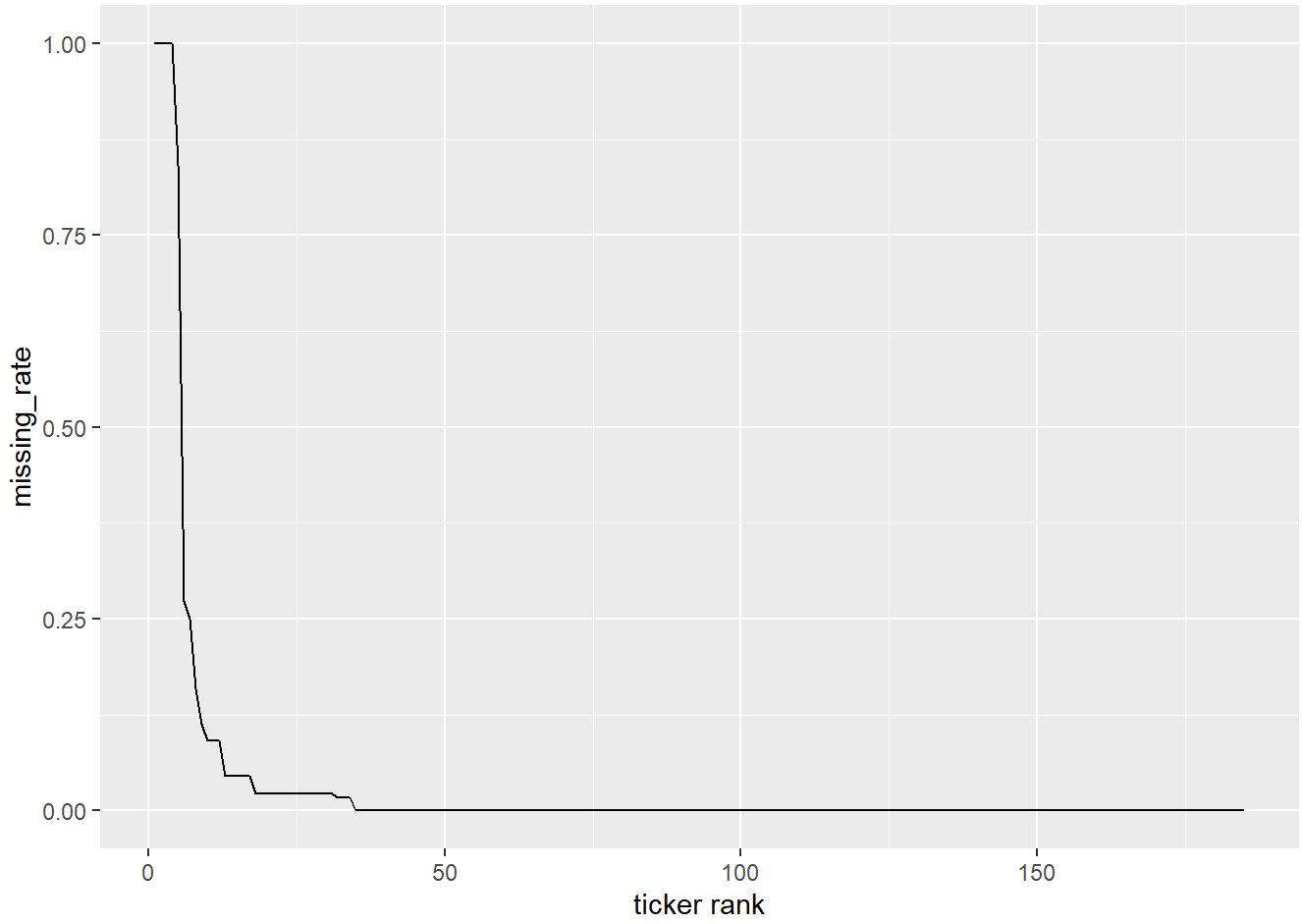
```
# 4) Ticker with highest transcript missing values vs total quarterly cells
symbol_rows <- transcripts_filtered_df %>%
  group_by(Symbol) %>%
  summarise(n_rows = n(), .groups = "drop")

missing_cells_df <- transcripts_filtered_df %>%
  group_by(Symbol) %>%
  summarise(missing_cells = sum(is.na(across(all_of(quarter_cols)))), .groups = "drop")

ticker_missing_summary <- symbol_rows %>%
  inner_join(missing_cells_df, by = "Symbol") %>%
  mutate(
    total_quarterly_cells = n_rows * length(quarter_cols),
    missing_rate = missing_cells / total_quarterly_cells
  ) %>%
  arrange(desc(missing_cells))

cat("Top 10 tickers by missing quarterly transcript values:\n")
```

```
## Top 10 tickers by missing quarterly transcript values:
```

```
print(head(ticker_missing_summary, 20))
```

```
## # A tibble: 20 × 5
##    Symbol n_rows missing_cells total_quarterly_cells missing_rate
##    <chr>   <int>         <int>                 <int>        <dbl>
##  1 COR         4           176                   176       1
##  2 CVX         4           176                   176       1
##  3 EXPD        4           176                   176       1
##  4 SCHW        4           176                   176       1
##  5 TROW        4           148                   176       0.841
##  6 MKC         4            48                   176       0.273
##  7 CTAS        4            44                   176       0.25
##  8 HUM         4            28                   176       0.159
##  9 WMT         4            20                   176       0.114
## 10 CI          4            16                   176       0.0909
## 11 COST        4            16                   176       0.0909
## 12 MU          4            16                   176       0.0909
## 13 AVGO        4             8                   176       0.0455
## 14 DRI         4             8                   176       0.0455
## 15 EXC         4             8                   176       0.0455
## 16 GIS         4             8                   176       0.0455
## 17 LIN         4             8                   176       0.0455
## 18 AEP         4             4                   176       0.0227
## 19 AKAM        4             4                   176       0.0227
## 20 ALL         4             4                   176       0.0227
```

```
ggplot(ticker_missing_summary, aes(x = seq_along(missing_rate), y = missing_rate)) +
  geom_line() +
  labs(x = "ticker rank", y = "missing_rate")
```




# Tickers with heavy quarterly missingness

Some tickers had **entirely** or **mostly** missing `CQ*` transcript fields. The following chunks **drop rows** for the top four tickers by missing quarterly cells, then re-check missing counts.

```
# 5) Remove rows for the top 4 tickers with the most missing quarterly transcript values
top4_missing_tickers <- head(ticker_missing_summary$Symbol, 4)
cat("Dropping rows for top 4 missing tickers:", paste0("['", paste(top4_missing_tickers, collaps
e = "', '"), "']"), "\n")
```

```
## Dropping rows for top 4 missing tickers: ['COR', 'CVX', 'EXPD', 'SCHW']
```

```
rows_before <- nrow(transcripts_filtered_df)
syms_before <- length(unique(transcripts_filtered_df$Symbol))

transcripts_filtered_df <- transcripts_filtered_df %>%
  filter(!Symbol %in% top4_missing_tickers)

cat("Rows before drop:", rows_before, "| after:", nrow(transcripts_filtered_df), "\n")
```

```
## Rows before drop: 740 | after: 724
```

```
cat("Unique symbols before/after:", syms_before, "|", length(unique(transcripts_filtered_df$Symb
ol)), "\n")
```

```
## Unique symbols before/after: 185 | 181
```

```
# 6) New missing values after deleting top 4 missing tickers
new_missing_per_quarter <- colSums(is.na(transcripts_filtered_df[, quarter_cols]))
cat("New missing values per quarter after deletion:\n")
```

```
## New missing values per quarter after deletion:
```

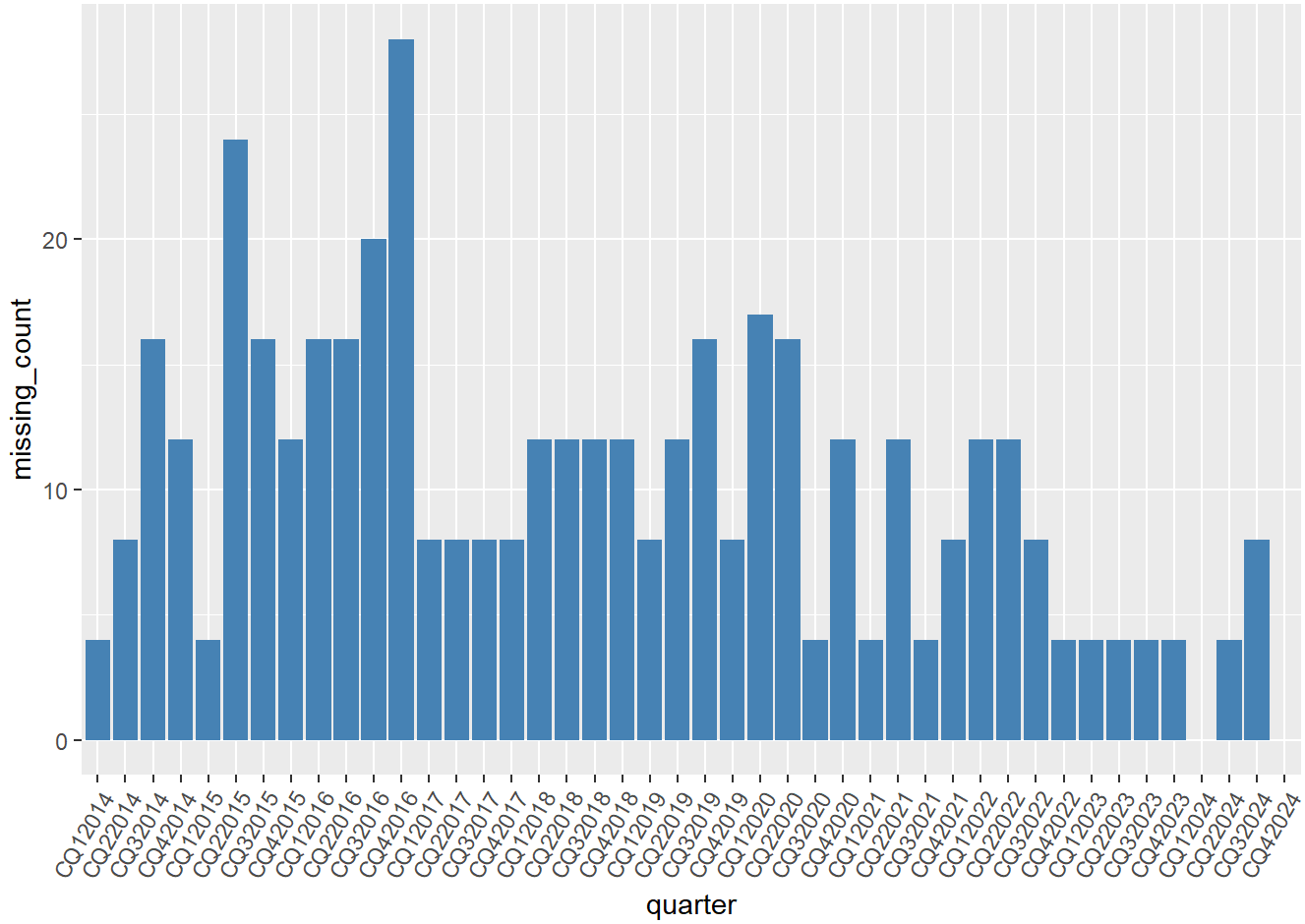
```
print(new_missing_per_quarter)
```

```
## CQ12014 CQ22014 CQ32014 CQ42014 CQ12015 CQ22015 CQ32015 CQ42015 CQ12016 CQ22016
##       4       8      16      12       4      24      16      12      16      16
## CQ32016 CQ42016 CQ12017 CQ22017 CQ32017 CQ42017 CQ12018 CQ22018 CQ32018 CQ42018
##      20      28       8       8       8       8      12      12      12      12
## CQ12019 CQ22019 CQ32019 CQ42019 CQ12020 CQ22020 CQ32020 CQ42020 CQ12021 CQ22021
##       8      12      16       8      17      16       4      12       4      12
## CQ32021 CQ42021 CQ12022 CQ22022 CQ32022 CQ42022 CQ12023 CQ22023 CQ32023 CQ42023
##       4       8      12      12       8       4       4       4       4       4
## CQ12024 CQ22024 CQ32024 CQ42024
##       0       4       8       0
```

```
cat("\nTotal missing quarterly values after deletion:", sum(new_missing_per_quarter), "\n")
```

```
##
## Total missing quarterly values after deletion: 441
```

```
new_missing_df <- tibble(
  quarter = factor(quarter_cols, levels = quarter_cols),
  missing_count = as.integer(new_missing_per_quarter)
)

ggplot(new_missing_df, aes(x = quarter, y = missing_count)) +
  geom_col(fill = "steelblue") +
  theme(axis.text.x = element_text(angle = 60, hjust = 1))
```




# After dropping high-missing transcript rows

Quarterly missing counts and a bar chart summarize remaining gaps after removing the worst tickers by missing `CQ*` cells.

```
# 7) Average and median missing rate across quarters in transcripts_df
quarter_cols_all <- grep("^CQ", colnames(transcripts_df), value = TRUE)

quarter_missing_rate <- colMeans(is.na(transcripts_df[, quarter_cols_all])) * 100

cat("Missing rate (%) of each quarter:\n")
```

```
## Missing rate (%) of each quarter:
```

```
print(quarter_missing_rate)
```

```
##  CQ12014  CQ22014  CQ32014  CQ42014  CQ12015  CQ22015  CQ32015  CQ42015
## 2.590674 3.108808 4.145078 3.626943 2.590674 5.181347 4.145078 3.626943
##  CQ12016  CQ22016  CQ32016  CQ42016  CQ12017  CQ22017  CQ32017  CQ42017
## 4.145078 4.145078 4.663212 5.699482 3.108808 3.108808 3.108808 3.108808
##  CQ12018  CQ22018  CQ32018  CQ42018  CQ12019  CQ22019  CQ32019  CQ42019
## 3.626943 3.626943 3.626943 3.626943 3.108808 3.626943 4.145078 3.108808
##  CQ12020  CQ22020  CQ32020  CQ42020  CQ12021  CQ22021  CQ32021  CQ42021
## 4.274611 4.145078 2.590674 3.626943 2.590674 3.626943 2.590674 3.108808
##  CQ12022  CQ22022  CQ32022  CQ42022  CQ12023  CQ22023  CQ32023  CQ42023
## 3.626943 3.626943 3.108808 2.590674 2.590674 2.590674 2.590674 2.590674
##  CQ12024  CQ22024  CQ32024  CQ42024
## 2.072539 2.590674 3.108808 2.072539
```

```
avg_missing_rate <- mean(quarter_missing_rate)
median_missing_rate <- median(quarter_missing_rate)

cat(sprintf("\nAverage missing rate across quarters: %.2f%%\n", avg_missing_rate))
```

```
##
## Average missing rate across quarters: 3.37%
```

```
cat(sprintf("Median missing rate across quarters: %.2f%%\n", median_missing_rate))
```

```
## Median missing rate across quarters: 3.11%
```

# 5. CRSP universe alignment and column prep

Drop tickers removed from transcripts, inspect CRSP uniqueness, **drop unused columns**, and parse `dlycaldt` as datetimes for time-series operations.

```
# 8) Remove from CRSP the same tickers removed from transcripts
removed_from_transcripts <- c()
if (exists("missing_symbols")) removed_from_transcripts <- c(removed_from_transcripts, missing_s
ymbols)
if (exists("top4_missing_tickers")) removed_from_transcripts <- c(removed_from_transcripts, top4
_missing_tickers)
removed_from_transcripts <- sort(unique(removed_from_transcripts))
cat("Tickers removed from transcripts_df:", paste0("['", paste(removed_from_transcripts, collaps
e = "', '"), "']"), "\n")
```

```
## Tickers removed from transcripts_df: ['ACN', 'AON', 'BF.B', 'BRK.B', 'COR', 'CVX', 'EQR', 'ET
N', 'EXPD', 'IVZ', 'SCHW', 'SLB']
```

```
crsp_rows_before <- nrow(crsp_df)
crsp_tickers_before <- length(unique(crsp_df$ticker))

crsp_df <- crsp_df %>% filter(!ticker %in% removed_from_transcripts)

cat("CRSP rows before/after:", crsp_rows_before, "|", nrow(crsp_df), "\n")
```

```
## CRSP rows before/after: 495987 | 485335
```

```
cat("CRSP unique tickers before/after:", crsp_tickers_before, "|", length(unique(crsp_df$ticke
r)), "\n")
```

```
## CRSP unique tickers before/after: 185 | 181
```

```
# --- CRSP sample rows after alignment ---
head(crsp_df)
```

```
## # A tibble: 6 × 27
##   permno permco ticker cusip   issuernm siccd  naics dlycaldt   dlyclose dlyopen
##    <dbl>  <dbl> <chr>  <chr>   <chr>    <dbl>  <dbl> <date>        <dbl>   <dbl>
## 1  10104   8045 ORCL   68389X… ORACLE …  7372 511210 2014-01-02     37.8    37.8
## 2  10104   8045 ORCL   68389X… ORACLE …  7372 511210 2014-01-03     37.6    37.6
## 3  10104   8045 ORCL   68389X… ORACLE …  7372 511210 2014-01-06     37.5    37.6
## 4  10104   8045 ORCL   68389X… ORACLE …  7372 511210 2014-01-07     37.8    37.7
## 5  10104   8045 ORCL   68389X… ORACLE …  7372 511210 2014-01-08     37.7    37.8
## 6  10104   8045 ORCL   68389X… ORACLE …  7372 511210 2014-01-09     37.6    37.8
## # ℹ 17 more variables: dlyhigh <dbl>, dlylow <dbl>, dlyprc <dbl>, dlyret <dbl>,
## #   dlyretx <dbl>, dlyvol <dbl>, shrout <dbl>, dlycap <dbl>, dlycumfacpr <dbl>,
## #   dlycumfacshr <dbl>, sprtrn <dbl>, vwretd <dbl>, ewretd <dbl>,
## #   securitytype <chr>, sharetype <chr>, usincflg <chr>, shradrflg <chr>
```

```
# --- CRSP cardinality of key columns ---
sapply(crsp_df, function(col) length(unique(col)))
```

```
##       permno       permco       ticker        cusip     issuernm        siccd
##          183          181          181          193          199          153
##        naics     dlycaldt     dlyclose      dlyopen      dlyhigh       dlylow
##          199         2768        63715        64618       102023       101723
##       dlyprc       dlyret      dlyretx       dlyvol       shrout       dlycap
##        63826        83766        83824       469692        10962       467669
##  dlycumfacpr dlycumfacshr       sprtrn       vwretd       ewretd securitytype
##           44           16         2640         2643         2644            1
##    sharetype     usincflg    shradrflg
##            1            1            1
```

```
# 10) Drop selected identifier/metadata columns from crsp_df
cols_to_drop <- c(
  "permno", "permco", "cusip", "issuernm",
  "securitytype", "sharetype", "usincflg", "shradrflg"
)

existing_cols_to_drop <- intersect(cols_to_drop, colnames(crsp_df))

crsp_df <- crsp_df %>% select(-all_of(existing_cols_to_drop))

cat("Dropped columns from crsp_df:", paste0("['", paste(existing_cols_to_drop, collapse = "',
'"), "']"), "\n")
```

```
## Dropped columns from crsp_df: ['permno', 'permco', 'cusip', 'issuernm', 'securitytype', 'shar
etype', 'usincflg', 'shradrflg']
```

```
cat("New crsp_df shape:", paste(nrow(crsp_df), ncol(crsp_df), sep = ", "), "\n")
```

```
## New crsp_df shape: 485335, 19
```

```
# Convert dlycaldt to Date dtype
crsp_df <- crsp_df %>% mutate(dlycaldt = ymd(dlycaldt))
```

```
# --- Transcript panel shape after filtering ---
dim(transcripts_filtered_df)
```

```
## [1] 724  48
```

# 6. Transcript reshape (ticker × quarter)

Pivot wide transcript data into **one row per (ticker, fiscal quarter)** with the modeling metrics as columns.

```
# 11) Reshape transcripts into one row per ticker-quarter with 4 metric columns
source_transcripts_df <- if (exists("transcripts_filtered_df")) transcripts_filtered_df else tra
nscripts_df

source_transcripts_df <- source_transcripts_df %>%
  rename(ticker = Symbol) %>%
  mutate(ticker = str_trim(as.character(ticker)))

metric_col_candidates <- colnames(source_transcripts_df)[tolower(str_trim(colnames(source_transc
ripts_df))) == "metrics"]
metric_col <- metric_col_candidates[1]
colnames(source_transcripts_df)[colnames(source_transcripts_df) == metric_col] <- "metric_name"
source_transcripts_df$metric_name <- str_trim(as.character(source_transcripts_df$metric_name))

metric_map <- c(
  "net positivity" = "net_positivity",
  "language complexity" = "language_complexity",
  "numeric transeprency" = "numeric_transparency",
  "numeric transparency" = "numeric_transparency",
  "analyst selectivity ratio" = "analyst_selectivity_ratio"
)

source_transcripts_df <- source_transcripts_df %>%
  mutate(metric_key = str_replace_all(tolower(metric_name), "\\s+", " "),
         metric_key = unname(metric_map[metric_key]))

quarter_cols <- grep("^CQ", colnames(source_transcripts_df), value = TRUE)
quarter_cols <- quarter_cols[order(sapply(quarter_cols, function(c) {
  as.integer(substr(c, 4, nchar(c))) * 10 + as.integer(substr(c, 3, 3))
}))]

id_cols <- c("ticker", "metric_key")

transcripts_long_df <- source_transcripts_df %>%
  select(all_of(c(id_cols, quarter_cols))) %>%
  pivot_longer(cols = all_of(quarter_cols),
               names_to = "quarter_code",
               values_to = "metric_value")

quarter_parts <- str_match(transcripts_long_df$quarter_code, "^CQ([1-4])(\\d{4})$")
transcripts_long_df$quarter_num <- as.integer(quarter_parts[, 2])
transcripts_long_df$year <- as.integer(quarter_parts[, 3])
transcripts_long_df <- transcripts_long_df %>%
  filter(!is.na(quarter_num) & !is.na(year))
transcripts_long_df$quarter_key <- as.yearqtr(paste0(transcripts_long_df$year, " Q", transcripts
_long_df$quarter_num))

transcripts_metric_df <- transcripts_long_df %>%
  select(ticker, quarter_key, metric_key, metric_value) %>%
  pivot_wider(names_from = metric_key, values_from = metric_value)

metric_feature_cols <- c(
```

```
  "net_positivity",
  "language_complexity",
  "numeric_transparency",
  "analyst_selectivity_ratio"
)
for (col in metric_feature_cols) {
  if (!col %in% colnames(transcripts_metric_df)) {
    transcripts_metric_df[[col]] <- NA
  }
}

transcripts_metric_df <- transcripts_metric_df %>%
  select(ticker, quarter_key, all_of(metric_feature_cols))

cat("transcripts_long_df shape:", paste(nrow(transcripts_long_df), ncol(transcripts_long_df), se
p = ", "), "\n")
```

```
## transcripts_long_df shape: 31856, 7
```

```
cat("transcripts_metric_df shape:", paste(nrow(transcripts_metric_df), ncol(transcripts_metric_d
f), sep = ", "), "\n")
```

```
## transcripts_metric_df shape: 7964, 6
```

```
head(transcripts_metric_df)
```

```
## # A tibble: 6 × 6
##   ticker quarter_key net_positivity language_complexity numeric_transparency
##   <chr>  <yearqtr>   <chr>          <chr>               <chr>
## 1 T      2014 Q1     1.54           11.12               2.39
## 2 T      2014 Q2     1.86           11.26               2.35
## 3 T      2014 Q3     1.85           11.11               2.31
## 4 T      2014 Q4     1.36           11.18               2.50
## 5 T      2015 Q1     1.14           11.43               1.97
## 6 T      2015 Q2     1.54           11.69               2.05
## # ℹ 1 more variable: analyst_selectivity_ratio <chr>
```

```
# --- Preview ticker-quarter metric table ---
head(transcripts_metric_df, 50)
```

```
## # A tibble: 50 × 6
##    ticker quarter_key net_positivity language_complexity numeric_transparency
##    <chr>  <yearqtr>   <chr>          <chr>               <chr>
##  1 T      2014 Q1     1.54           11.12               2.39
##  2 T      2014 Q2     1.86           11.26               2.35
##  3 T      2014 Q3     1.85           11.11               2.31
##  4 T      2014 Q4     1.36           11.18               2.50
##  5 T      2015 Q1     1.14           11.43               1.97
##  6 T      2015 Q2     1.54           11.69               2.05
##  7 T      2015 Q3     2.01           11.69               1.70
##  8 T      2015 Q4     1.81           12.05               1.56
##  9 T      2016 Q1     1.50           11.35               1.93
## 10 T      2016 Q2     2.24           11.71               1.84
## # ℹ 40 more rows
## # ℹ 1 more variable: analyst_selectivity_ratio <chr>
```

# 7. Daily panel merge and target

Join CRSP daily rows to the **prior completed quarter** transcript metrics, **deduplicate** `(ticker, date)` on CRSP and again after merge, build **`next_day_dlyretx`** (primary modeling target) and **`next_day_return`** (diagnostic, from `dlyret`), audit/drop rare non-last target NaNs, validate metric coverage.

```
# 12) Build ML panel — merge CRSP daily with prior-quarter transcript metrics
crsp_merge_df <- crsp_df %>%
  mutate(ticker = str_trim(as.character(ticker)),
         dlycaldt = ymd(dlycaldt)) %>%
  arrange(ticker, dlycaldt) %>%
  distinct(ticker, dlycaldt, .keep_all = TRUE)

# Derive quarter keys: current quarter and the prior completed quarter (avoids look-ahead bias)
crsp_merge_df <- crsp_merge_df %>%
  mutate(current_quarter_key = as.yearqtr(dlycaldt),
         prior_quarter_key = current_quarter_key - 0.25)

metric_feature_cols <- c(
  "net_positivity",
  "language_complexity",
  "numeric_transparency",
  "analyst_selectivity_ratio"
)

transcript_join_df <- transcripts_metric_df %>%
  rename(prior_quarter_key = quarter_key) %>%
  mutate(ticker = str_trim(as.character(ticker)))

merged_daily_df <- crsp_merge_df %>%
  left_join(transcript_join_df, by = c("ticker", "prior_quarter_key"))

# Transcript metrics arrive as strings from pivot; cast to numeric for modeling
for (col in metric_feature_cols) {
  merged_daily_df[[col]] <- suppressWarnings(as.numeric(merged_daily_df[[col]]))
}

rows_pre_md <- nrow(merged_daily_df)
merged_daily_df <- merged_daily_df %>%
  arrange(ticker, dlycaldt) %>%
  distinct(ticker, dlycaldt, .keep_all = TRUE)
cat("Post-merge dedup removed", rows_pre_md - nrow(merged_daily_df), "duplicate (ticker, dlycald
t) rows\n")
```

```
## Post-merge dedup removed 0 duplicate (ticker, dlycaldt) rows
```

```
cat("crsp_merge_df shape :", paste(nrow(crsp_merge_df), ncol(crsp_merge_df), sep = ", "), "\n")
```

```
## crsp_merge_df shape : 482551, 21
```

```
cat("merged_daily_df shape:", paste(nrow(merged_daily_df), ncol(merged_daily_df), sep = ", "),
"\n")
```

```
## merged_daily_df shape: 482551, 25
```

```
head(merged_daily_df)
```

```
## # A tibble: 6 × 25
##   ticker siccd  naics dlycaldt   dlyclose dlyopen dlyhigh dlylow dlyprc   dlyret
##   <chr>  <dbl>  <dbl> <date>        <dbl>   <dbl>   <dbl>  <dbl>  <dbl>    <dbl>
## 1 A       3825 334515 2014-01-02     56.2    57.1    57.1   56.2   56.2 -1.71e-2
## 2 A       3825 334515 2014-01-03     56.9    56.4    57.3   56.3   56.9  1.26e-2
## 3 A       3825 334515 2014-01-06     56.6    57.4    57.7   56.6   56.6 -4.92e-3
## 4 A       3825 334515 2014-01-07     57.4    57.0    57.6   56.9   57.4  1.43e-2
## 5 A       3825 334515 2014-01-08     58.4    57.3    58.5   57.2   58.4  1.64e-2
## 6 A       3825 334515 2014-01-09     58.4    58.4    58.7   57.9   58.4  3.43e-4
## # ℹ 15 more variables: dlyretx <dbl>, dlyvol <dbl>, shrout <dbl>, dlycap <dbl>,
## #   dlycumfacpr <dbl>, dlycumfacshr <dbl>, sprtrn <dbl>, vwretd <dbl>,
## #   ewretd <dbl>, current_quarter_key <yearqtr>, prior_quarter_key <yearqtr>,
## #   net_positivity <dbl>, language_complexity <dbl>,
## #   numeric_transparency <dbl>, analyst_selectivity_ratio <dbl>
```

```
# --- Merged daily panel: dtypes and non-null counts ---
glimpse(merged_daily_df)
```

```
## Rows: 482,551
## Columns: 25
## $ ticker                   <chr> "A", "A", "A", "A", "A", "A", "A", "A", "A",…
## $ siccd                    <dbl> 3825, 3825, 3825, 3825, 3825, 3825, 3825, 38…
## $ naics                    <dbl> 334515, 334515, 334515, 334515, 334515, 3345…
## $ dlycaldt                 <date> 2014-01-02, 2014-01-03, 2014-01-06, 2014-01…
## $ dlyclose                 <dbl> 56.21, 56.92, 56.64, 57.45, 58.39, 58.41, 58…
## $ dlyopen                  <dbl> 57.10, 56.39, 57.40, 56.95, 57.33, 58.40, 58…
## $ dlyhigh                  <dbl> 57.1000, 57.3450, 57.7000, 57.6300, 58.5400,…
## $ dlylow                   <dbl> 56.150, 56.260, 56.560, 56.930, 57.170, 57.8…
## $ dlyprc                   <dbl> 56.21, 56.92, 56.64, 57.45, 58.39, 58.41, 58…
## $ dlyret                   <dbl> -0.017136, 0.012631, -0.004919, 0.014301, 0.…
## $ dlyretx                  <dbl> -0.017136, 0.012631, -0.004919, 0.014301, 0.…
## $ dlyvol                   <dbl> 1916200, 1866700, 1777500, 1463200, 2659500,…
## $ shrout                   <dbl> 331809, 331809, 331809, 331809, 331809, 3318…
## $ dlycap                   <dbl> 18650984, 18886568, 18793662, 19062427, 1937…
## $ dlycumfacpr              <dbl> 1.381366, 1.381366, 1.381366, 1.381366, 1.38…
## $ dlycumfacshr             <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,…
## $ sprtrn                   <dbl> -0.008862, -0.000333, -0.002512, 0.006082, -…
## $ vwretd                   <dbl> -0.008757, 0.000491, -0.003340, 0.006090, 0.…
## $ ewretd                   <dbl> -0.004051, 0.004096, -0.001676, 0.006892, 0.…
## $ current_quarter_key      <yearqtr> 2014 Q1, 2014 Q1, 2014 Q1, 2014 Q1, 2014…
## $ prior_quarter_key        <yearqtr> 2013 Q4, 2013 Q4, 2013 Q4, 2013 Q4, 2013…
## $ net_positivity           <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, …
## $ language_complexity      <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, …
## $ numeric_transparency     <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, …
## $ analyst_selectivity_ratio <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, …
```

```
cat("\nNon-null counts:\n")
```

```
##
## Non-null counts:
```

```
print(colSums(!is.na(merged_daily_df)))
```

```
##                    ticker                     siccd                     naics
##                    482551                    482551                    482551
##                  dlycaldt                  dlyclose                   dlyopen
##                    482551                    482549                    482549
##                   dlyhigh                    dlylow                    dlyprc
##                    482549                    482549                    482550
##                    dlyret                   dlyretx                    dlyvol
##                    482548                    482548                    482550
##                    shrout                    dlycap              dlycumfacpr
##                    482551                    482550                    482551
##               dlycumfacshr                    sprtrn                    vwretd
##                    482551                    482551                    482551
##                    ewretd       current_quarter_key         prior_quarter_key
##                    482551                    482551                    482551
##            net_positivity       language_complexity      numeric_transparency
##                    462586                    462586                    462586
## analyst_selectivity_ratio
##                    462775
```

```
# 13) Primary model target: next_day_dlyretx (next row dlyretx). next_day_return is diagnostic
(next row dlyret).
merged_daily_df <- merged_daily_df %>%
  arrange(ticker, dlycaldt) %>%
  group_by(ticker) %>%
  mutate(
    next_day_return = dplyr::lead(dlyret),
    next_day_dlyretx = dplyr::lead(dlyretx)
  ) %>%
  ungroup()

cat("next_day_return null count:", sum(is.na(merged_daily_df$next_day_return)), "\n")
```

```
## next_day_return null count: 183
```

```
cat("next_day_dlyretx null count:", sum(is.na(merged_daily_df$next_day_dlyretx)), "\n")
```

```
## next_day_dlyretx null count: 183
```

```
cat("Unique tickers (expect same NaN count for last-day rows):", length(unique(merged_daily_df$t
icker)), "\n")
```

```
## Unique tickers (expect same NaN count for last-day rows): 181
```

```
head(
  merged_daily_df %>%
    select(ticker, dlycaldt, dlyret, dlyretx, next_day_return, next_day_dlyretx),
  10
)
```

```
## # A tibble: 10 × 6
##    ticker dlycaldt      dlyret   dlyretx next_day_return next_day_dlyretx
##    <chr>  <date>         <dbl>     <dbl>           <dbl>            <dbl>
##  1 A      2014-01-02 -0.0171   -0.0171          0.0126           0.0126
##  2 A      2014-01-03  0.0126    0.0126         -0.00492         -0.00492
##  3 A      2014-01-06 -0.00492  -0.00492         0.0143           0.0143
##  4 A      2014-01-07  0.0143    0.0143          0.0164           0.0164
##  5 A      2014-01-08  0.0164    0.0164          0.000343         0.000343
##  6 A      2014-01-09  0.000343  0.000343        0.00890          0.00890
##  7 A      2014-01-10  0.00890   0.00890         0                0
##  8 A      2014-01-13  0         0               0.0161           0.0161
##  9 A      2014-01-14  0.0161    0.0161          0.00768          0.00768
## 10 A      2014-01-15  0.00768   0.00768         0.00265          0.00265
```

```
# --- Data quality checks on merged panel (metrics vs returns) ---
# 14) Validation checks
metric_feature_cols <- c(
  "net_positivity",
  "language_complexity",
  "numeric_transparency",
  "analyst_selectivity_ratio"
)

# 1. Row preservation
crsp_row_count <- nrow(crsp_merge_df)
merged_row_count <- nrow(merged_daily_df)
cat(sprintf("Row preservation — CRSP: %s  |  Merged: %s\n",
            format(crsp_row_count, big.mark = ","),
            format(merged_row_count, big.mark = ",")))
```

```
## Row preservation — CRSP: 482,551  |  Merged: 482,551
```

```
stopifnot(crsp_row_count == merged_row_count)

# 2. Key uniqueness / duplicate-inflation check
crsp_dup_keys <- sum(duplicated(crsp_merge_df[, c("ticker", "dlycaldt")]))
dup_keys <- sum(duplicated(merged_daily_df[, c("ticker", "dlycaldt")]))
cat(sprintf("Duplicate (ticker, dlycaldt) in CRSP source : %d\n", crsp_dup_keys))
```

```
## Duplicate (ticker, dlycaldt) in CRSP source : 0
```

```
cat(sprintf("Duplicate (ticker, dlycaldt) after merge  : %d\n", dup_keys))
```

```
## Duplicate (ticker, dlycaldt) after merge  : 0
```

```
stopifnot(dup_keys == crsp_dup_keys)

# 3. Metric missingness after merge
missing_rates <- round(colMeans(is.na(merged_daily_df[, metric_feature_cols])) * 100, 2)
cat("\nMetric missing rate (%) after merge:\n")
```

```
##
## Metric missing rate (%) after merge:
```

```
print(missing_rates)
```

```
##            net_positivity       language_complexity      numeric_transparency
##                      4.14                      4.14                      4.14
## analyst_selectivity_ratio
##                      4.10
```

```
# 4–4b) next_day_return / next_day_dlyretx NaN audit — should only be NaN at last trading day pe
r ticker

audit_one_target <- function(df, tgt_col, px_col) {
  n_t <- length(unique(df$ticker))
  last_day_per_ticker <- df %>%
    arrange(ticker, dlycaldt) %>%
    group_by(ticker) %>%
    summarise(dlycaldt = last(dlycaldt), .groups = "drop")
  last_day_join <- last_day_per_ticker %>%
    inner_join(df %>% select(ticker, dlycaldt, all_of(tgt_col)),
               by = c("ticker", "dlycaldt"))
  last_day_nulls <- sum(is.na(last_day_join[[tgt_col]]))
  non_last_nulls <- sum(is.na(df[[tgt_col]])) - last_day_nulls
  cat(sprintf("\n%s NaN at last date per ticker: %d / %d\n", tgt_col, last_day_nulls, n_t))
  cat(sprintf("%s NaN at non-last dates (unexpected): %d\n", tgt_col, non_last_nulls))

  evt <- df %>%
    arrange(ticker, dlycaldt) %>%
    group_by(ticker) %>%
    mutate(
      is_last_trade = dlycaldt == max(dlycaldt),
      px_lead = dplyr::lead(.data[[px_col]]),
      dlycaldt_lead = dplyr::lead(dlycaldt)
    ) %>%
    ungroup()

  anom <- evt %>% filter(is.na(.data[[tgt_col]]), !is_last_trade)
  cat(sprintf("\n=== %s anomaly audit (non-last NaN) ===\n", tgt_col))
  cat("Anomaly row count:", nrow(anom), "\n")
  if (nrow(anom) > 0) {
    show_cols <- c("ticker", "dlycaldt", px_col, "px_lead", tgt_col, "dlycaldt_lead")
    print(anom %>% select(any_of(show_cols)))
    cat("Interpretation: ", tgt_col, " should match ", px_col, " on next trade date; NaN usually
means missing ",
        px_col, " on next trade date.\n", sep = "")
  }
  anom %>% distinct(ticker, dlycaldt)
}

drop_keys <- NULL
drop_keys <- bind_rows(
  audit_one_target(merged_daily_df, "next_day_return", "dlyret"),
  audit_one_target(merged_daily_df, "next_day_dlyretx", "dlyretx")
)
```

```
##
## next_day_return NaN at last date per ticker: 181 / 181
## next_day_return NaN at non-last dates (unexpected): 2
##
## === next_day_return anomaly audit (non-last NaN) ===
## Anomaly row count: 2
## # A tibble: 2 × 6
##   ticker dlycaldt      dlyret px_lead next_day_return dlycaldt_lead
##   <chr>  <date>         <dbl>   <dbl>           <dbl> <date>
## 1 BIIB   2020-11-05 -0.0752        NA              NA 2020-11-06
## 2 GOOG   2014-04-02  0.000185      NA              NA 2014-04-03
## Interpretation: next_day_return should match dlyret on next trade date; NaN usually means mis
sing dlyret on next trade date.
##
## next_day_dlyretx NaN at last date per ticker: 181 / 181
## next_day_dlyretx NaN at non-last dates (unexpected): 2
##
## === next_day_dlyretx anomaly audit (non-last NaN) ===
## Anomaly row count: 2
## # A tibble: 2 × 6
##   ticker dlycaldt     dlyretx px_lead next_day_dlyretx dlycaldt_lead
##   <chr>  <date>         <dbl>   <dbl>            <dbl> <date>
## 1 BIIB   2020-11-05 -0.0752        NA               NA 2020-11-06
## 2 GOOG   2014-04-02  0.000185      NA               NA 2014-04-03
## Interpretation: next_day_dlyretx should match dlyretx on next trade date; NaN usually means m
issing dlyretx on next trade date.
```

```
if (!is.null(drop_keys) && nrow(drop_keys) > 0) {
  cat("\nDropping these rows (reason code TARGET_NEXT_NA).\n")
  merged_daily_df <- merged_daily_df %>%
    anti_join(drop_keys, by = c("ticker", "dlycaldt")) %>%
    arrange(ticker, dlycaldt) %>%
    group_by(ticker) %>%
    mutate(
      next_day_return = dplyr::lead(dlyret),
      next_day_dlyretx = dplyr::lead(dlyretx)
    ) %>%
    ungroup()

  cat("\n--- After drop ---\n")
  audit_one_target(merged_daily_df, "next_day_return", "dlyret")
  audit_one_target(merged_daily_df, "next_day_dlyretx", "dlyretx")
}
```

```
##
## Dropping these rows (reason code TARGET_NEXT_NA).
##
## --- After drop ---
##
## next_day_return NaN at last date per ticker: 181 / 181
## next_day_return NaN at non-last dates (unexpected): 2
##
## === next_day_return anomaly audit (non-last NaN) ===
## Anomaly row count: 2
## # A tibble: 2 × 6
##   ticker dlycaldt   dlyret px_lead next_day_return dlycaldt_lead
##   <chr>  <date>      <dbl>   <dbl>           <dbl> <date>
## 1 BIIB   2020-11-04 0.440       NA              NA 2020-11-06
## 2 GOOG   2014-04-01 0.0183      NA              NA 2014-04-03
## Interpretation: next_day_return should match dlyret on next trade date; NaN usually means mis
sing dlyret on next trade date.
##
## next_day_dlyretx NaN at last date per ticker: 181 / 181
## next_day_dlyretx NaN at non-last dates (unexpected): 2
##
## === next_day_dlyretx anomaly audit (non-last NaN) ===
## Anomaly row count: 2
## # A tibble: 2 × 6
##   ticker dlycaldt   dlyretx px_lead next_day_dlyretx dlycaldt_lead
##   <chr>  <date>       <dbl>   <dbl>            <dbl> <date>
## 1 BIIB   2020-11-04  0.440       NA               NA 2020-11-06
## 2 GOOG   2014-04-01  0.0183      NA               NA 2014-04-03
## Interpretation: next_day_dlyretx should match dlyretx on next trade date; NaN usually means m
issing dlyretx on next trade date.
```

```
## # A tibble: 2 × 2
##   ticker dlycaldt
##   <chr>  <date>
## 1 BIIB   2020-11-04
## 2 GOOG   2014-04-01
```

```
# 5. Spot-check: quarter alignment for one ticker
sample_ticker <- merged_daily_df$ticker[1]
cat(sprintf("\nSpot-check for %s — date, current_quarter, prior_quarter, net_positivity:\n", sam
ple_ticker))
```

```
##
## Spot-check for A — date, current_quarter, prior_quarter, net_positivity:
```

```
print(head(merged_daily_df %>%
             filter(ticker == sample_ticker) %>%
             select(dlycaldt, current_quarter_key, prior_quarter_key, net_positivity), 10))
```

```
## # A tibble: 10 × 4
##    dlycaldt   current_quarter_key prior_quarter_key net_positivity
##    <date>     <yearqtr>           <yearqtr>                  <dbl>
##  1 2014-01-02 2014 Q1             2013 Q4                       NA
##  2 2014-01-03 2014 Q1             2013 Q4                       NA
##  3 2014-01-06 2014 Q1             2013 Q4                       NA
##  4 2014-01-07 2014 Q1             2013 Q4                       NA
##  5 2014-01-08 2014 Q1             2013 Q4                       NA
##  6 2014-01-09 2014 Q1             2013 Q4                       NA
##  7 2014-01-10 2014 Q1             2013 Q4                       NA
##  8 2014-01-13 2014 Q1             2013 Q4                       NA
##  9 2014-01-14 2014 Q1             2013 Q4                       NA
## 10 2014-01-15 2014 Q1             2013 Q4                       NA
```

```
# --- One row per (ticker, trade date) — idempotent (dedup already applied after merge) ---
merged_daily_df <- merged_daily_df %>%
  mutate(dlycaldt = ymd(as.character(dlycaldt)))

rows_before <- nrow(merged_daily_df)
merged_daily_df <- merged_daily_df %>%
  arrange(ticker, dlycaldt) %>%
  distinct(ticker, dlycaldt, .keep_all = TRUE)
rows_after <- nrow(merged_daily_df)

cat("Rows before:", rows_before, "\n")
```

```
## Rows before: 482549
```

```
cat("Rows after:", rows_after, "\n")
```

```
## Rows after: 482549
```

```
cat("Removed (expect 0 if upstream dedup succeeded):", rows_before - rows_after, "\n")
```

```
## Removed (expect 0 if upstream dedup succeeded): 0
```

```
# --- Merged daily panel: numeric summary ---
summary(merged_daily_df)
```

```
##    ticker              siccd          naics           dlycaldt
##  Length:482549      Min.   : 831   Min.   :113210   Min.   :2014-01-02
##  Class :character   1st Qu.:3550   1st Qu.:325620   1st Qu.:2016-10-13
##  Mode  :character   Median :4911   Median :339113   Median :2019-07-23
##                     Mean   :4722   Mean   :411091   Mean   :2019-07-14
##                     3rd Qu.:6211   3rd Qu.:522110   3rd Qu.:2022-04-13
##                     Max.   :8742   Max.   :812332   Max.   :2024-12-31
##
##    dlyclose          dlyopen           dlyhigh            dlylow
##  Min.   :   2.10   Min.   :   2.20   Min.   :   2.60   Min.   :   1.93
##  1st Qu.:  54.99   1st Qu.:  54.99   1st Qu.:  55.50   1st Qu.:  54.45
##  Median :  91.79   Median :  91.76   Median :  92.67   Median :  90.85
##  Mean   : 162.03   Mean   : 162.01   Mean   : 163.68   Mean   : 160.30
##  3rd Qu.: 163.55   3rd Qu.: 163.56   3rd Qu.: 165.14   3rd Qu.: 161.97
##  Max.   :5300.34   Max.   :5300.00   Max.   :5337.24   Max.   :5260.00
##  NA's   :2         NA's   :2         NA's   :2         NA's   :2
##     dlyprc            dlyret             dlyretx               dlyvol
##  Min.   :   2.10   Min.   :-0.538647   Min.   :-0.5386470   Min.   :        0
##  1st Qu.:  54.99   1st Qu.:-0.007401   1st Qu.:-0.0075160   1st Qu.:  1319782
##  Median :  91.79   Median : 0.000758   Median : 0.0006800   Median :  2683912
##  Mean   : 162.03   Mean   : 0.000599   Mean   : 0.0005096   Mean   :  6521085
##  3rd Qu.: 163.55   3rd Qu.: 0.008800   3rd Qu.: 0.0087350   3rd Qu.:  6234047
##  Max.   :5300.34   Max.   : 0.646465   Max.   : 0.6464650   Max.   :657542346
##  NA's   :1         NA's   :3           NA's   :3            NA's   :1
##     shrout             dlycap            dlycumfacpr       dlycumfacshr
##  Min.   :    9875   Min.   :     38249   Min.   : 0.2022   Min.   : 0.125
##  1st Qu.:  243654   1st Qu.:  22372751   1st Qu.: 1.0000   1st Qu.: 1.000
##  Median :  461341   Median :  44861972   Median : 1.0000   Median : 1.000
##  Mean   : 1066010   Mean   : 102912328   Mean   : 2.0087   Mean   : 1.996
##  3rd Qu.:  980137   3rd Qu.: 100995883   3rd Qu.: 1.0000   3rd Qu.: 1.000
##  Max.   :24568840   Max.   :3915300473   Max.   :70.0000   Max.   :70.000
##                     NA's   :1
##     sprtrn               vwretd               ewretd
##  Min.   :-0.1198410   Min.   :-0.1181680   Min.   :-0.1076310
##  1st Qu.:-0.0037530   1st Qu.:-0.0038390   1st Qu.:-0.0045750
##  Median : 0.0006480   Median : 0.0007010   Median : 0.0007110
##  Mean   : 0.0004793   Mean   : 0.0004856   Mean   : 0.0003608
##  3rd Qu.: 0.0056750   3rd Qu.: 0.0056540   3rd Qu.: 0.0057600
##  Max.   : 0.0938280   Max.   : 0.0915560   Max.   : 0.0821750
##
##  current_quarter_key prior_quarter_key net_positivity   language_complexity
##  Min.   :2014        Min.   :2014      Min.   :-1.430   Min.   : 9.05
##  1st Qu.:2017        1st Qu.:2016      1st Qu.: 0.740   1st Qu.:11.63
##  Median :2020        Median :2019      Median : 1.110   Median :12.39
##  Mean   :2019        Mean   :2019      Mean   : 1.114   Mean   :12.43
##  3rd Qu.:2022        3rd Qu.:2022      3rd Qu.: 1.500   3rd Qu.:13.15
##  Max.   :2025        Max.   :2024      Max.   : 3.320   Max.   :17.74
##                                        NA's   :19964    NA's   :19964
##  numeric_transparency analyst_selectivity_ratio next_day_return
##  Min.   :0.290        Min.   :  0.00            Min.   :-0.538647
##  1st Qu.:1.800        1st Qu.: 30.43            1st Qu.:-0.007396
##  Median :2.260        Median : 41.18            Median : 0.000761
```

```
##  Mean   :2.342        Mean   : 42.17            Mean   : 0.000603
##  3rd Qu.:2.800        3rd Qu.: 53.33            3rd Qu.: 0.008803
##  Max.   :7.460        Max.   :100.00            Max.   : 0.646465
##  NA's   :19964        NA's   :19775             NA's   :183
##  next_day_dlyretx
##  Min.   :-0.538647
##  1st Qu.:-0.007509
##  Median : 0.000684
##  Mean   : 0.000513
##  3rd Qu.: 0.008739
##  Max.   : 0.646465
##  NA's   :183
```

# 8. Feature engineering and time-based split

- **Staged features (`FE_STAGE` 1–5)**: (1) baseline transcript levels + `dlyret` lags/rolls + market OHLCV; (2) add `dlyretx` **lags and rolling mean/std**; (3) add **OHLC microstructure** (overnight gap, intraday return, high–low range, Garman–Klass); (4) add **transcript dynamics** (QoQ deltas, deviation vs trailing 2-quarter mean, days since transcript quarter change); (5) add **cross-sectional ranks/z** by date and **industry-relative** `dlyretx` vs same-day `siccd` median.
- **Scaling**: **z-score** transcript level + dynamics columns with **train-only** means/SDs (non-finite values imputed at train means before scaling, matching `StandardScaler` behavior).
- **Governance**: duplicate-key assert and train/val calendar ordering check inside the feature chunk.
- **Split**: train / validation / test by calendar end dates (unchanged).

**Experimental protocol:** set `FE_STAGE` to `1 … 5` in the chunk below, then re-run the feature-engineering chunk, then the split + model chunks; compare RMSE / MAE / R² / directional accuracy / Spearman IC on the held-out test window. Default `FE_STAGE = 5` is the full pipeline.

```
# Run A / Run B benchmark control
# A = strict no-transcript predictors
# B = full transcript pipeline
RUN_LABELS <- c("A", "B")
RUN_NAMES <- c(
  A = "Strict no-transcript",
  B = "Full transcript"
)
```

```
# Optional ablation control: set FE_STAGE to 1..5 before running feature engineering.
# 1 = baseline, 2 = +dlyretx lag/roll, 3 = +OHLC, 4 = +transcript dynamics, 5 = +cross-section (full).
FE_STAGE <- 5L
```

```
# ── Step 1: Feature engineering (mirrors Final_code.ipynb) ──────────────────
# Staged ablation: 1=baseline (levels+miss+market+dlyret lags/roll+cal+ids),
# 2=+dlyretx lag/rolling, 3=+OHLC microstructure, 4=+transcript dynamics, 5=+cross-section (ful
l)
FE_STAGE <- if (exists("FE_STAGE")) as.integer(FE_STAGE[1]) else 5L
if (!FE_STAGE %in% 1L:5L) stop("FE_STAGE must be between 1 and 5")

model_df <- merged_daily_df %>%
  mutate(dlycaldt = ymd(as.character(dlycaldt))) %>%
  arrange(ticker, dlycaldt)

TRANSCRIPT_FEATS <- c(
  "net_positivity", "language_complexity",
  "numeric_transparency", "analyst_selectivity_ratio"
)

# Raw transcript snapshot (before scaling) for missingness + dynamics source
tr_raw <- model_df[, TRANSCRIPT_FEATS, drop = FALSE]

# ── Transcript dynamics (quarter-level) ─────────────────────────────────────
TRANSCRIPT_DYN_FEATS <- character(0)
if (FE_STAGE >= 4L && "prior_quarter_key" %in% names(model_df)) {
  dyn_base <- model_df %>%
    distinct(ticker, prior_quarter_key, .keep_all = TRUE) %>%
    arrange(ticker, prior_quarter_key)

  for (m in TRANSCRIPT_FEATS) {
    dyn_base <- dyn_base %>%
      group_by(ticker) %>%
      mutate(
        !!paste0(m, "_dq") := .data[[m]] - dplyr::lag(.data[[m]], 1L),
        !!paste0(m, "_dev2") := .data[[m]] - (dplyr::lag(.data[[m]], 1L) + dplyr::lag(.data
[[m]], 2L)) / 2
      ) %>%
      ungroup()
  }
  dyn_join_cols <- c(
    "ticker", "prior_quarter_key",
    unlist(lapply(TRANSCRIPT_FEATS, function(m) c(paste0(m, "_dq"), paste0(m, "_dev2"))))
  )
  dyn_join_cols <- intersect(dyn_join_cols, names(dyn_base))
  model_df <- model_df %>%
    left_join(dyn_base %>% select(any_of(dyn_join_cols)), by = c("ticker", "prior_quarter_key"))

  model_df <- model_df %>%
    arrange(ticker, dlycaldt) %>%
    group_by(ticker) %>%
    mutate(
      q_prev = dplyr::lag(prior_quarter_key, 1L),
      ep_inc = dplyr::case_when(
        is.na(q_prev) ~ 1L,
        prior_quarter_key != q_prev ~ 1L,
```

```
        TRUE ~ 0L
      ),
      txn_ep = cumsum(ep_inc)
    ) %>%
    group_by(ticker, txn_ep) %>%
    mutate(days_since_transcript_change = as.integer(dlycaldt - min(dlycaldt))) %>%
    ungroup() %>%
    select(-q_prev, -ep_inc, -txn_ep)

  TRANSCRIPT_DYN_FEATS <- c(
    unlist(lapply(TRANSCRIPT_FEATS, function(m) c(paste0(m, "_dq"), paste0(m, "_dev2")))),
    "days_since_transcript_change"
  )
} else {
  for (m in TRANSCRIPT_FEATS) {
    model_df[[paste0(m, "_dq")]] <- NA_real_
    model_df[[paste0(m, "_dev2")]] <- NA_real_
  }
  model_df[["days_since_transcript_change"]] <- NA_real_
  TRANSCRIPT_DYN_FEATS <- c(
    unlist(lapply(TRANSCRIPT_FEATS, function(m) c(paste0(m, "_dq"), paste0(m, "_dev2")))),
    "days_since_transcript_change"
  )
}

# ── Lagged dlyret ──────────────────────────────────────────────────────
for (lag_k in c(1L, 2L, 5L, 10L, 20L)) {
  model_df <- model_df %>%
    group_by(ticker) %>%
    mutate(!!paste0("dlyret_lag", lag_k) := dplyr::lag(dlyret, lag_k)) %>%
    ungroup()
}

# ── dlyretx lag + rolling (FE_STAGE >= 2) ──────────────────────────────
RETX_LAG_FEATS <- paste0("dlyretx_lag", c(1, 2, 5, 10, 20))
RETX_ROLL_FEATS <- c(
  paste0("retx_mean_", c(5, 10, 20)),
  paste0("retx_std_", c(5, 10, 20))
)
if (FE_STAGE >= 2L) {
  for (lag_k in c(1L, 2L, 5L, 10L, 20L)) {
    model_df <- model_df %>%
      group_by(ticker) %>%
      mutate(!!paste0("dlyretx_lag", lag_k) := dplyr::lag(dlyretx, lag_k)) %>%
      ungroup()
  }
  for (w in c(5L, 10L, 20L)) {
    model_df <- model_df %>%
      group_by(ticker) %>%
      arrange(dlycaldt, .by_group = TRUE) %>%
      mutate(
        !!paste0("retx_mean_", w) := slider::slide_dbl(
```

```
          dplyr::lag(dlyretx, 1L), mean,
          .before = w - 1L, .complete = FALSE, na.rm = TRUE
        ),
        !!paste0("retx_std_", w) := slider::slide_dbl(
          dplyr::lag(dlyretx, 1L), stats::sd,
          .before = w - 1L, .complete = FALSE, na.rm = TRUE
        )
      ) %>%
      ungroup()
  }
} else {
  for (nm in RETX_LAG_FEATS) model_df[[nm]] <- NA_real_
  for (nm in RETX_ROLL_FEATS) model_df[[nm]] <- NA_real_
}

# ── Rolling stats on dlyret / dlyvol (shift by 1; windows 5 and 10) ──────────
for (window in c(5L, 10L)) {
  model_df <- model_df %>%
    group_by(ticker) %>%
    arrange(dlycaldt, .by_group = TRUE) %>%
    mutate(
      !!paste0("ret_mean_", window) := slider::slide_dbl(
        dplyr::lag(dlyret, 1L), mean,
        .before = window - 1L, .complete = FALSE, na.rm = TRUE
      ),
      !!paste0("ret_std_", window) := slider::slide_dbl(
        dplyr::lag(dlyret, 1L), stats::sd,
        .before = window - 1L, .complete = FALSE, na.rm = TRUE
      ),
      !!paste0("vol_mean_", window) := slider::slide_dbl(
        dplyr::lag(dlyvol, 1L), mean,
        .before = window - 1L, .complete = FALSE, na.rm = TRUE
      )
    ) %>%
    ungroup()
}

# ── OHLC microstructure ──────────────────────────────────────────────────────
MICROSTRUCT_FEATS <- c("overnight_gap", "intraday_return", "high_low_range", "gk_vol")
safe_div <- function(a, b, eps = 1e-12) {
  b2 <- ifelse(!is.na(b) & abs(b) < eps, sign(b) * eps, b)
  b2 <- ifelse(!is.na(b2) & b2 == 0, NA_real_, b2)
  ifelse(is.na(b2), NA_real_, a / b2)
}

model_df <- model_df %>%
  group_by(ticker) %>%
  mutate(dlyclose_lag1 = dplyr::lag(dlyclose, 1L)) %>%
  ungroup()

if (FE_STAGE >= 3L) {
  model_df <- model_df %>%
```

```
    mutate(
      overnight_gap = safe_div(dlyopen, dlyclose_lag1) - 1,
      intraday_return = safe_div(dlyclose, dlyopen) - 1,
      high_low_range = safe_div(dlyhigh - dlylow, ifelse(dlyclose == 0, NA_real_, dlyclose)),
      gk_vol = {
        lo <- log(safe_div(dlyhigh, dlylow))
        lc <- log(safe_div(dlyclose, dlyopen))
        0.5 * (lo^2) - (2 * log(2) - 1) * (lc^2)
      }
    )
} else {
  for (nm in MICROSTRUCT_FEATS) model_df[[nm]] <- NA_real_
}

# ── Cross-section + industry residual (FE_STAGE >= 5) ──────────────────────
XSEC_FEATS <- character(0)
if (FE_STAGE >= 5L) {
  xsec_numeric <- c("dlyretx", "dlyret", "ret_mean_10", "ret_std_10", "vol_mean_10")
  for (xc in xsec_numeric) {
    if (!xc %in% names(model_df)) next
    model_df <- model_df %>%
      group_by(dlycaldt) %>%
      mutate(
        !!paste0(xc, "_cs_rank") := dplyr::percent_rank(.data[[xc]]),
        !!paste0(xc, "_cs_z") := {
          v <- .data[[xc]]
          m <- mean(v, na.rm = TRUE)
          s <- stats::sd(v, na.rm = TRUE)
          if (!is.finite(s) || s < 1e-12) rep(NA_real_, dplyr::n()) else (v - m) / s
        }
      ) %>%
      ungroup()
    XSEC_FEATS <- c(XSEC_FEATS, paste0(xc, "_cs_rank"), paste0(xc, "_cs_z"))
  }
  if ("siccd" %in% names(model_df)) {
    model_df <- model_df %>%
      group_by(dlycaldt, siccd) %>%
      mutate(dlyretx_ind_med = stats::median(dlyretx, na.rm = TRUE)) %>%
      ungroup() %>%
      mutate(dlyretx_ind_res = dlyretx - dlyretx_ind_med)
  } else {
    model_df$dlyretx_ind_med <- NA_real_
    model_df$dlyretx_ind_res <- NA_real_
  }
  XSEC_FEATS <- c(XSEC_FEATS, "dlyretx_ind_med", "dlyretx_ind_res")
} else {
  for (xc in c("dlyretx", "dlyret", "ret_mean_10", "ret_std_10", "vol_mean_10")) {
    model_df[[paste0(xc, "_cs_rank")]] <- NA_real_
    model_df[[paste0(xc, "_cs_z")]] <- NA_real_
    XSEC_FEATS <- c(XSEC_FEATS, paste0(xc, "_cs_rank"), paste0(xc, "_cs_z"))
  }
  model_df$dlyretx_ind_med <- NA_real_
```

```
  model_df$dlyretx_ind_res <- NA_real_
  XSEC_FEATS <- c(XSEC_FEATS, "dlyretx_ind_med", "dlyretx_ind_res")
}

# ── Calendar features (pandas dayofweek: Monday=0) ──────────────────────
model_df <- model_df %>%
  mutate(
    month = lubridate::month(dlycaldt),
    dayofweek = (lubridate::wday(dlycaldt, week_start = 1) - 1L) %% 7L
  )

# ── Label-encode categorical identifiers (R tree APIs) ──────────────────
for (col in c("ticker", "siccd", "naics")) {
  model_df[[paste0(col, "_enc")]] <- as.integer(factor(as.character(model_df[[col]]))) - 1L
}

# Missingness indicators from raw transcript NA
for (tf in TRANSCRIPT_FEATS) {
  model_df[[paste0("miss_", tf)]] <- as.integer(is.na(tr_raw[[tf]]))
}
model_df$miss_any_transcript <- as.integer(pmax(
  model_df$miss_net_positivity,
  model_df$miss_language_complexity,
  model_df$miss_numeric_transparency,
  model_df$miss_analyst_selectivity_ratio
))
TRANSCRIPT_MISS_FEATS <- c(paste0("miss_", TRANSCRIPT_FEATS), "miss_any_transcript")

# Train-only scaling on transcript levels + dynamics (when FE_STAGE >= 4)
TRAIN_END_SCALER <- as.Date("2021-12-31")
scaler_cols <- if (FE_STAGE >= 4L) {
  c(TRANSCRIPT_FEATS, intersect(TRANSCRIPT_DYN_FEATS, names(model_df)))
} else {
  TRANSCRIPT_FEATS
}

train_fit_mask <- model_df$dlycaldt <= TRAIN_END_SCALER
train_sub <- model_df[train_fit_mask, scaler_cols, drop = FALSE]
train_sub <- train_sub[stats::complete.cases(train_sub), , drop = FALSE]
if (nrow(train_sub) < 10) {
  scaler_cols <- TRANSCRIPT_FEATS
  train_sub <- model_df[train_fit_mask, scaler_cols, drop = FALSE]
  train_sub <- train_sub[stats::complete.cases(train_sub), , drop = FALSE]
}

mu_vec <- colMeans(train_sub, na.rm = TRUE)
sig_vec <- apply(train_sub, 2, stats::sd, na.rm = TRUE)
sig_vec[!is.finite(sig_vec) | sig_vec == 0] <- 1

for (nm in scaler_cols) {
  x <- as.numeric(model_df[[nm]])
  x[!is.finite(x)] <- mu_vec[[nm]]
```

```
  model_df[[nm]] <- (x - mu_vec[[nm]]) / sig_vec[[nm]]
}

MARKET_FEATS <- c("dlyret", "dlyretx", "dlyvol", "dlycap", "dlyclose", "dlyopen", "dlyhigh", "dl
ylow")
LAG_FEATS <- paste0("dlyret_lag", c(1, 2, 5, 10, 20))
ROLLING_FEATS <- c(
  paste0("ret_mean_", c(5, 10)),
  paste0("ret_std_", c(5, 10)),
  paste0("vol_mean_", c(5, 10))
)
CALENDAR_FEATS <- c("month", "dayofweek")
ID_FEATS <- c("ticker_enc", "siccd_enc", "naics_enc")

FEATURE_COLS <- c(
  TRANSCRIPT_FEATS, TRANSCRIPT_MISS_FEATS, MARKET_FEATS,
  LAG_FEATS, ROLLING_FEATS
)
if (FE_STAGE >= 2L) FEATURE_COLS <- c(FEATURE_COLS, RETX_LAG_FEATS, RETX_ROLL_FEATS)
if (FE_STAGE >= 3L) FEATURE_COLS <- c(FEATURE_COLS, MICROSTRUCT_FEATS)
if (FE_STAGE >= 4L) FEATURE_COLS <- c(FEATURE_COLS, intersect(TRANSCRIPT_DYN_FEATS, names(model_
df)))
if (FE_STAGE >= 5L) FEATURE_COLS <- c(FEATURE_COLS, intersect(XSEC_FEATS, names(model_df)))
FEATURE_COLS <- c(FEATURE_COLS, CALENDAR_FEATS, ID_FEATS)
FEATURE_COLS <- intersect(FEATURE_COLS, names(model_df))

# ── Run A / Run B feature sets (strict benchmark) ──────────────────────────
FEATURE_COLS_B <- FEATURE_COLS
dyn_cols_present <- intersect(TRANSCRIPT_DYN_FEATS, names(model_df))
TRANSCRIPT_EXCLUDE_COLS <- unique(c(
  TRANSCRIPT_FEATS,
  TRANSCRIPT_MISS_FEATS,
  dyn_cols_present
))
FEATURE_COLS_A <- setdiff(FEATURE_COLS_B, TRANSCRIPT_EXCLUDE_COLS)

RUN_FEATURE_SETS <- list(
  A = FEATURE_COLS_A,
  B = FEATURE_COLS_B
)
if (!exists("RUN_LABELS")) RUN_LABELS <- c("A", "B")
RUN_LABELS <- intersect(RUN_LABELS, names(RUN_FEATURE_SETS))
if (!length(RUN_LABELS)) stop("No valid RUN_LABELS found in RUN_FEATURE_SETS.")

# Backward-compatible default references use full model
FEATURE_COLS <- FEATURE_COLS_B

TARGET_COL <- "next_day_dlyretx"
model_df <- model_df %>% filter(!is.na(.data[[TARGET_COL]]))

stopifnot(!anyDuplicated(model_df[, c("ticker", "dlycaldt")]))
train_max <- max(model_df$dlycaldt[model_df$dlycaldt <= TRAIN_END_SCALER], na.rm = TRUE)
```

```
val_min <- min(model_df$dlycaldt[model_df$dlycaldt >= as.Date("2022-01-01")], na.rm = TRUE)
stopifnot(train_max < val_min)

cat(sprintf(
  "FE_STAGE=%d (1=baseline, 2=+dlyretx lag/roll, 3=+OHLC, 4=+transcript dyn, 5=+xsec/full)\n",
  FE_STAGE
))
```

```
## FE_STAGE=5 (1=baseline, 2=+dlyretx lag/roll, 3=+OHLC, 4=+transcript dyn, 5=+xsec/full)
```

```
cat(sprintf("model_df shape : %s\n", paste(nrow(model_df), ncol(model_df), sep = ", ")))
```

```
## model_df shape : 482366, 85
```

```
cat(sprintf("Features (%d): %s\n", length(FEATURE_COLS),
            paste0("['", paste(FEATURE_COLS, collapse = "', '"), "']")))
```

```
## Features (69): ['net_positivity', 'language_complexity', 'numeric_transparency', 'analyst_sel
ectivity_ratio', 'miss_net_positivity', 'miss_language_complexity', 'miss_numeric_transparency',
'miss_analyst_selectivity_ratio', 'miss_any_transcript', 'dlyret', 'dlyretx', 'dlyvol', 'dlyca
p', 'dlyclose', 'dlyopen', 'dlyhigh', 'dlylow', 'dlyret_lag1', 'dlyret_lag2', 'dlyret_lag5', 'dl
yret_lag10', 'dlyret_lag20', 'ret_mean_5', 'ret_mean_10', 'ret_std_5', 'ret_std_10', 'vol_mean_
5', 'vol_mean_10', 'dlyretx_lag1', 'dlyretx_lag2', 'dlyretx_lag5', 'dlyretx_lag10', 'dlyretx_lag
20', 'retx_mean_5', 'retx_mean_10', 'retx_mean_20', 'retx_std_5', 'retx_std_10', 'retx_std_20',
'overnight_gap', 'intraday_return', 'high_low_range', 'gk_vol', 'net_positivity_dq', 'net_positi
vity_dev2', 'language_complexity_dq', 'language_complexity_dev2', 'numeric_transparency_dq', 'nu
meric_transparency_dev2', 'analyst_selectivity_ratio_dq', 'analyst_selectivity_ratio_dev2', 'day
s_since_transcript_change', 'dlyretx_cs_rank', 'dlyretx_cs_z', 'dlyret_cs_rank', 'dlyret_cs_z',
'ret_mean_10_cs_rank', 'ret_mean_10_cs_z', 'ret_std_10_cs_rank', 'ret_std_10_cs_z', 'vol_mean_10
_cs_rank', 'vol_mean_10_cs_z', 'dlyretx_ind_med', 'dlyretx_ind_res', 'month', 'dayofweek', 'tick
er_enc', 'siccd_enc', 'naics_enc']
```

```
cat(sprintf("Target         : %s\n", TARGET_COL))
```

```
## Target         : next_day_dlyretx
```

```
cat("\nTranscript missingness rate (% of days) — indicator means:\n")
```

```
##
## Transcript missingness rate (% of days) — indicator means:
```

```
print(round(colMeans(model_df[, TRANSCRIPT_MISS_FEATS, drop = FALSE]) * 100, 2))
```

```
##            miss_net_positivity       miss_language_complexity
##                           4.14                           4.14
##      miss_numeric_transparency miss_analyst_selectivity_ratio
##                           4.14                           4.10
##            miss_any_transcript
##                           4.14
```

```
dyn_cols <- intersect(TRANSCRIPT_DYN_FEATS, names(model_df))
cat("\nTranscript + dynamics — NaN % after scaling (expect ~0 on scaled cols):\n")
```

```
##
## Transcript + dynamics — NaN % after scaling (expect ~0 on scaled cols):
```

```
print(round(colMeans(is.na(model_df[, c(TRANSCRIPT_FEATS, dyn_cols), drop = FALSE])) * 100, 4))
```

```
##                 net_positivity            language_complexity
##                              0                              0
##           numeric_transparency      analyst_selectivity_ratio
##                              0                              0
##              net_positivity_dq            net_positivity_dev2
##                              0                              0
##         language_complexity_dq       language_complexity_dev2
##                              0                              0
##        numeric_transparency_dq      numeric_transparency_dev2
##                              0                              0
##   analyst_selectivity_ratio_dq analyst_selectivity_ratio_dev2
##                              0                              0
##   days_since_transcript_change
##                              0
```

```
cat("\nMissing rates (%) per feature:\n")
```

```
##
## Missing rates (%) per feature:
```

```
print(round(colMeans(is.na(model_df[, FEATURE_COLS, drop = FALSE])) * 100, 2))
```

```
##                 net_positivity            language_complexity
##                           0.00                           0.00
##           numeric_transparency      analyst_selectivity_ratio
##                           0.00                           0.00
##            miss_net_positivity       miss_language_complexity
##                           0.00                           0.00
##      miss_numeric_transparency miss_analyst_selectivity_ratio
##                           0.00                           0.00
##            miss_any_transcript                         dlyret
##                           0.00                           0.00
##                         dlyretx                         dlyvol
##                           0.00                           0.00
##                          dlycap                        dlyclose
##                           0.00                           0.00
##                         dlyopen                         dlyhigh
##                           0.00                           0.00
##                          dlylow                     dlyret_lag1
##                           0.00                           0.04
##                     dlyret_lag2                     dlyret_lag5
##                           0.08                           0.19
##                    dlyret_lag10                    dlyret_lag20
##                           0.38                           0.75
##                      ret_mean_5                     ret_mean_10
##                           0.04                           0.04
##                       ret_std_5                      ret_std_10
##                           0.08                           0.08
##                      vol_mean_5                     vol_mean_10
##                           0.04                           0.04
##                    dlyretx_lag1                    dlyretx_lag2
##                           0.04                           0.08
##                    dlyretx_lag5                   dlyretx_lag10
##                           0.19                           0.38
##                   dlyretx_lag20                     retx_mean_5
##                           0.75                           0.04
##                    retx_mean_10                    retx_mean_20
##                           0.04                           0.04
##                      retx_std_5                     retx_std_10
##                           0.08                           0.08
##                     retx_std_20                    overnight_gap
##                           0.08                           0.04
##                intraday_return                   high_low_range
##                           0.00                           0.00
##                          gk_vol                net_positivity_dq
##                           0.00                           0.00
##             net_positivity_dev2           language_complexity_dq
##                           0.00                           0.00
##        language_complexity_dev2          numeric_transparency_dq
##                           0.00                           0.00
##      numeric_transparency_dev2     analyst_selectivity_ratio_dq
##                           0.00                           0.00
## analyst_selectivity_ratio_dev2     days_since_transcript_change
##                           0.00                           0.00
```

```
##                dlyretx_cs_rank                    dlyretx_cs_z
##                           0.00                            0.00
##                 dlyret_cs_rank                     dlyret_cs_z
##                           0.00                            0.00
##            ret_mean_10_cs_rank                ret_mean_10_cs_z
##                           0.04                            0.04
##             ret_std_10_cs_rank                 ret_std_10_cs_z
##                           0.08                            0.08
##            vol_mean_10_cs_rank                vol_mean_10_cs_z
##                           0.04                            0.04
##                dlyretx_ind_med                dlyretx_ind_res
##                           0.00                            0.00
##                          month                       dayofweek
##                           0.00                            0.00
##                     ticker_enc                       siccd_enc
##                           0.00                            0.00
##                      naics_enc
##                           0.00
```

```
cat("\nRun feature counts:\n")
```

```
##
## Run feature counts:
```

```
for (run_id in RUN_LABELS) {
  cat(sprintf("  Run %s (%s): %d features\n",
              run_id,
              if (exists("RUN_NAMES") && run_id %in% names(RUN_NAMES)) RUN_NAMES[[run_id]] else
run_id,
              length(RUN_FEATURE_SETS[[run_id]])))
}
```

```
##   Run A (Strict no-transcript): 51 features
##   Run B (Full transcript): 69 features
```

```
strict_overlap <- intersect(RUN_FEATURE_SETS$A, TRANSCRIPT_EXCLUDE_COLS)
cat(sprintf("Run A transcript-overlap feature count (expect 0): %d\n", length(strict_overlap)))
```

```
## Run A transcript-overlap feature count (expect 0): 0
```

```
stopifnot(length(strict_overlap) == 0L)
```

```
# --- Modeling matrix preview ---
head(model_df)
```

```
## # A tibble: 6 × 85
##   ticker siccd  naics dlycaldt   dlyclose dlyopen dlyhigh dlylow dlyprc   dlyret
##   <chr>  <dbl>  <dbl> <date>        <dbl>   <dbl>   <dbl>  <dbl>  <dbl>    <dbl>
## 1 A       3825 334515 2014-01-02     56.2    57.1    57.1   56.2   56.2 -1.71e-2
## 2 A       3825 334515 2014-01-03     56.9    56.4    57.3   56.3   56.9  1.26e-2
## 3 A       3825 334515 2014-01-06     56.6    57.4    57.7   56.6   56.6 -4.92e-3
## 4 A       3825 334515 2014-01-07     57.4    57.0    57.6   56.9   57.4  1.43e-2
## 5 A       3825 334515 2014-01-08     58.4    57.3    58.5   57.2   58.4  1.64e-2
## 6 A       3825 334515 2014-01-09     58.4    58.4    58.7   57.9   58.4  3.43e-4
## # ℹ 75 more variables: dlyretx <dbl>, dlyvol <dbl>, shrout <dbl>, dlycap <dbl>,
## #   dlycumfacpr <dbl>, dlycumfacshr <dbl>, sprtrn <dbl>, vwretd <dbl>,
## #   ewretd <dbl>, current_quarter_key <yearqtr>, prior_quarter_key <yearqtr>,
## #   net_positivity <dbl>, language_complexity <dbl>,
## #   numeric_transparency <dbl>, analyst_selectivity_ratio <dbl>,
## #   next_day_return <dbl>, next_day_dlyretx <dbl>, net_positivity_dq <dbl>,
## #   net_positivity_dev2 <dbl>, language_complexity_dq <dbl>, …
```

```
# ── Step 2: Time-based train / validation / test split ──────────────────────
TRAIN_END  <- as.Date("2021-12-31")
VAL_START  <- as.Date("2022-01-01")
VAL_END    <- as.Date("2022-12-31")
TEST_START <- as.Date("2023-01-01")

train_df <- model_df %>% filter(dlycaldt <= TRAIN_END)
val_df   <- model_df %>% filter(dlycaldt >= VAL_START & dlycaldt <= VAL_END)
test_df  <- model_df %>% filter(dlycaldt >= TEST_START)

# Sanity checks — no overlap
stopifnot(max(train_df$dlycaldt) < min(val_df$dlycaldt))
stopifnot(max(val_df$dlycaldt)   < min(test_df$dlycaldt))

cat(sprintf("Train rows      : %10s  (%s → %s)   (%s)\n",
            format(nrow(train_df), big.mark = ","),
            as.character(min(train_df$dlycaldt)),
            as.character(max(train_df$dlycaldt)),
            paste(nrow(train_df), ncol(train_df), sep = ", ")))
```

```
## Train rows      :    349,557  (2014-01-02 → 2021-12-31)   (349557, 85)
```

```
cat(sprintf("Validation rows : %10s  (%s → %s)\n",
            format(nrow(val_df), big.mark = ","),
            as.character(min(val_df$dlycaldt)),
            as.character(max(val_df$dlycaldt))))
```

```
## Validation rows :     44,318  (2022-01-03 → 2022-12-30)
```

```
cat(sprintf("Test rows       : %10s  (%s → %s)\n",
            format(nrow(test_df), big.mark = ","),
            as.character(min(test_df$dlycaldt)),
            as.character(max(test_df$dlycaldt))))
```

```
## Test rows       :     88,491  (2023-01-03 → 2024-12-30)
```

```
# Build per-run matrices from the same split boundaries/rows.
RUN_SPLITS <- list()
for (run_id in RUN_LABELS) {
  run_cols <- RUN_FEATURE_SETS[[run_id]]
  X_train <- train_df[, run_cols, drop = FALSE]
  y_train <- train_df[[TARGET_COL]]
  X_val   <- val_df[, run_cols, drop = FALSE]
  y_val   <- val_df[[TARGET_COL]]
  X_test  <- test_df[, run_cols, drop = FALSE]
  y_test  <- test_df[[TARGET_COL]]
  RUN_SPLITS[[run_id]] <- list(
    feature_cols = run_cols,
    X_train = X_train, y_train = y_train,
    X_val = X_val, y_val = y_val,
    X_test = X_test, y_test = y_test
  )
}

# Row-level comparability guardrail across runs.
row_counts <- vapply(RUN_SPLITS, function(x) c(
  train = nrow(x$X_train),
  val = nrow(x$X_val),
  test = nrow(x$X_test)
), numeric(3L))
stopifnot(length(unique(row_counts["train", ])) == 1L)
stopifnot(length(unique(row_counts["val", ])) == 1L)
stopifnot(length(unique(row_counts["test", ])) == 1L)

# Backward-compatible aliases point to Run B (or first available run).
default_run <- if ("B" %in% names(RUN_SPLITS)) "B" else names(RUN_SPLITS)[1]
X_train <- RUN_SPLITS[[default_run]]$X_train
y_train <- RUN_SPLITS[[default_run]]$y_train
X_val   <- RUN_SPLITS[[default_run]]$X_val
y_val   <- RUN_SPLITS[[default_run]]$y_val
X_test  <- RUN_SPLITS[[default_run]]$X_test
y_test  <- RUN_SPLITS[[default_run]]$y_test
```

# 9. Model training, tuning, and comparison

**LightGBM** and **XGBoost** baselines, optional Random Forest, shared **evaluation** helpers, **CPU** setup in this R port (the Python notebook probes **GPU** where available), **walk-forward expanding-window time-series CV** on train+validation (by `dlycaldt`), refit best parameters on full train+val, a **comparison table**, and **feature**

**importance** plots.

```
# ── Step 3: Evaluation helper ──────────────────────────────────────────
evaluate <- function(y_true, y_pred, label = "") {
  # Return a list of regression and financial metrics; optionally print them.
  y_true <- as.numeric(y_true)
  y_pred <- as.numeric(y_pred)

  ok <- !is.na(y_true) & !is.na(y_pred)
  yt <- y_true[ok]
  yp <- y_pred[ok]

  rmse <- sqrt(mean((yt - yp)^2))
  mae  <- mean(abs(yt - yp))
  ss_res <- sum((yt - yp)^2)
  ss_tot <- sum((yt - mean(yt))^2)
  r2  <- 1 - ss_res / ss_tot
  dir_acc <- mean(sign(yt) == sign(yp))
  ic <- suppressWarnings(cor(yt, yp, method = "spearman"))

  result <- list(
    RMSE = rmse,
    MAE = mae,
    R2 = r2,
    Directional_Accuracy = dir_acc,
    Spearman_IC = ic
  )
  if (nzchar(label)) {
    cat(sprintf("\n── %s %s\n", label, strrep("─", max(0, 50 - nchar(label)))))
    for (k in names(result)) {
      cat(sprintf("  %-25s: %f\n", k, result[[k]]))
    }
  }
  result
}

# Container filled by model chunks below
results <- list()
models_by_run <- list()
tuning_by_run <- list()

cat("evaluate() helper registered. results dict initialised.\n")
```

```
## evaluate() helper registered. results dict initialised.
```

```
# ── Step 4a: LightGBM ──────────────────────────────────────────────────
# Build lgb.Dataset with categorical_feature hint for label-encoded identifiers.
for (run_id in RUN_LABELS) {
  split_i <- RUN_SPLITS[[run_id]]
  X_train_mat <- data.matrix(split_i$X_train)
  X_val_mat   <- data.matrix(split_i$X_val)
  X_test_mat  <- data.matrix(split_i$X_test)
  cat_feats <- intersect(c("ticker_enc", "siccd_enc", "naics_enc"), colnames(split_i$X_train))

  dtrain_lgb <- lgb.Dataset(
    data = X_train_mat,
    label = split_i$y_train,
    categorical_feature = cat_feats
  )
  dval_lgb <- lgb.Dataset(
    data = X_val_mat,
    label = split_i$y_val,
    categorical_feature = cat_feats,
    reference = dtrain_lgb
  )

  lgb_params <- list(
    objective = "regression",
    learning_rate = 0.01,
    num_leaves = 127,
    min_data_in_leaf = 50,
    feature_fraction = 0.8,
    bagging_fraction = 0.8,
    max_depth = 6,
    verbose = -1
  )

  lgb_model <- lgb.train(
    params = lgb_params,
    data = dtrain_lgb,
    nrounds = 5000,
    valids = list(valid = dval_lgb),
    early_stopping_rounds = 100,
    eval = "l2"
  )

  lgb_preds <- predict(lgb_model, X_test_mat)
  key <- paste(run_id, "LightGBM", sep = "::")
  lbl <- sprintf("Run %s (%s) LightGBM — Test set (2023-2024)",
                 run_id,
                 if (exists("RUN_NAMES") && run_id %in% names(RUN_NAMES)) RUN_NAMES[[run_id]] el
se run_id)
  results[[key]] <- evaluate(split_i$y_test, lgb_preds, label = lbl)

  if (is.null(models_by_run[[run_id]])) models_by_run[[run_id]] <- list()
  models_by_run[[run_id]]$lgb_model <- lgb_model
  models_by_run[[run_id]]$cat_feats <- cat_feats
```

```
  models_by_run[[run_id]]$X_test_mat <- X_test_mat

  cat(sprintf("Run %s LightGBM best iteration: %s\n", run_id, lgb_model$best_iter))
}
```

```
##
## ── Run A (Strict no-transcript) LightGBM — Test set (2023-2024)
##   RMSE                    : 0.016408
##   MAE                     : 0.011238
##   R2                      : 0.000354
##   Directional_Accuracy    : 0.521206
##   Spearman_IC             : 0.003704
## Run A LightGBM best iteration: 28
##
## ── Run B (Full transcript) LightGBM — Test set (2023-2024)
##   RMSE                    : 0.016408
##   MAE                     : 0.011238
##   R2                      : 0.000406
##   Directional_Accuracy    : 0.521669
##   Spearman_IC             : -0.004059
## Run B LightGBM best iteration: 16
```

```
# ── Step 4b: XGBoost ──────────────────────────────────────────────────
# CPU-only: cast the three *_enc columns to numeric, build xgb.DMatrix.
if (!exists("xgb_tree_method")) {
  xgb_tree_method <- "hist"
}

for (run_id in RUN_LABELS) {
  split_i <- RUN_SPLITS[[run_id]]
  cat_feats <- intersect(c("ticker_enc", "siccd_enc", "naics_enc"), colnames(split_i$X_train))

  X_train_xgb <- split_i$X_train
  X_val_xgb   <- split_i$X_val
  X_test_xgb  <- split_i$X_test
  for (col in cat_feats) {
    X_train_xgb[[col]] <- as.numeric(X_train_xgb[[col]])
    X_val_xgb[[col]]   <- as.numeric(X_val_xgb[[col]])
    X_test_xgb[[col]]  <- as.numeric(X_test_xgb[[col]])
  }

  dtrain_xgb <- xgb.DMatrix(data = data.matrix(X_train_xgb), label = split_i$y_train)
  dval_xgb   <- xgb.DMatrix(data = data.matrix(X_val_xgb),   label = split_i$y_val)
  dtest_xgb  <- xgb.DMatrix(data = data.matrix(X_test_xgb),  label = split_i$y_test)

  xgb_params <- list(
    objective = "reg:squarederror",
    tree_method = xgb_tree_method,  # set in GPU stub below to "hist"
    eta = 0.01,
    max_depth = 6,
    subsample = 0.8,
    colsample_bytree = 0.8
  )

  xgb_model <- xgb.train(
    params = xgb_params,
    data = dtrain_xgb,
    nrounds = 20000,
    watchlist = list(val = dval_xgb),
    early_stopping_rounds = 100,
    verbose = 0
  )

  xgb_preds <- predict(xgb_model, dtest_xgb)
  key <- paste(run_id, "XGBoost", sep = "::")
  lbl <- sprintf("Run %s (%s) XGBoost — Test set (2023-2024)",
                 run_id,
                 if (exists("RUN_NAMES") && run_id %in% names(RUN_NAMES)) RUN_NAMES[[run_id]] el
se run_id)
  results[[key]] <- evaluate(split_i$y_test, xgb_preds, label = lbl)

  if (is.null(models_by_run[[run_id]])) models_by_run[[run_id]] <- list()
  models_by_run[[run_id]]$xgb_model <- xgb_model
  models_by_run[[run_id]]$X_train_xgb <- X_train_xgb
```

```
  models_by_run[[run_id]]$X_val_xgb <- X_val_xgb
  models_by_run[[run_id]]$X_test_xgb <- X_test_xgb
  models_by_run[[run_id]]$dtest_xgb <- dtest_xgb

  cat(sprintf("Run %s XGBoost best iteration: %s\n", run_id, xgb_model$best_iteration))
}
```

```
##
## ── Run A (Strict no-transcript) XGBoost — Test set (2023-2024)
##   RMSE                    : 0.016408
##   MAE                     : 0.011238
##   R2                      : 0.000390
##   Directional_Accuracy    : 0.520369
##   Spearman_IC             : 0.007655
```

```
##
## ── Run B (Full transcript) XGBoost — Test set (2023-2024)
##   RMSE                    : 0.016404
##   MAE                     : 0.011239
##   R2                      : 0.000798
##   Directional_Accuracy    : 0.519217
##   Spearman_IC             : 0.008264
```

```
# ── Tuning Setup: walk-forward time-series CV (mirrors Final_code.ipynb) ──────
# Expanding-window folds on `dlycaldt` over train+val only (pre-test).

TS_CV_N_FOLDS <- 4L
TS_CV_MIN_TRAIN_ROWS <- 5000L

array_split_vec <- function(x, k) {
  n <- length(x)
  if (k <= 0L) return(list())
  base <- n %/% k
  rem <- n %% k
  sizes <- rep(base, k)
  if (rem > 0L) sizes[seq_len(rem)] <- sizes[seq_len(rem)] + 1L
  starts <- c(1L, cumsum(utils::head(sizes, -1L)) + 1L)
  ends <- cumsum(sizes)
  lapply(seq_len(k), function(i) x[starts[i]:ends[i]])
}

make_expanding_date_folds <- function(d_trval, n_folds = TS_CV_N_FOLDS,
                                      min_train_rows = TS_CV_MIN_TRAIN_ROWS) {
  d <- as.Date(d_trval)
  u <- sort(unique(d))
  n_u <- length(u)
  n_folds <- as.integer(min(max(2L, n_folds), max(2L, n_u - 1L)))
  parts <- array_split_vec(u, n_folds + 1L)
  folds <- list()
  for (k in seq_len(n_folds)) {
    train_dates <- sort(unique(do.call(c, parts[seq_len(k)])))
    val_dates <- parts[[k + 1L]]
    if (!length(val_dates)) next
    tr_idx <- which(d %in% train_dates)
    va_idx <- which(d %in% val_dates)
    if (length(tr_idx) < min_train_rows || length(va_idx) == 0L) next
    stopifnot(max(d[tr_idx]) < min(d[va_idx]))
    folds[[length(folds) + 1L]] <- list(train = tr_idx, val = va_idx)
  }
  folds
}

summarize_ts_folds <- function(d_trval, folds, name) {
  d <- as.Date(d_trval)
  cat(sprintf("\n── %s: time-series folds (expanding by date) %s\n",
              name, strrep("─", max(0L, 12L))))
  for (i in seq_along(folds)) {
    tr <- folds[[i]]$train
    va <- folds[[i]]$val
    cat(sprintf(
      "  Fold %d: train %s rows  (%s → %s)  |  val %s rows  (%s → %s)\n",
      i,
      format(length(tr), big.mark = ","),
      as.character(min(d[tr])),
      as.character(max(d[tr])),
```

```
      format(length(va), big.mark = ","),
      as.character(min(d[va])),
      as.character(max(d[va]))
    ))
  }
}

run_context <- list()
for (run_id in RUN_LABELS) {
  split_i <- RUN_SPLITS[[run_id]]
  trval_df_i <- dplyr::bind_rows(train_df, val_df)
  d_trval_i <- trval_df_i$dlycaldt
  X_trval_i <- dplyr::bind_rows(split_i$X_train, split_i$X_val)
  y_trval_i <- c(split_i$y_train, split_i$y_val)
  stopifnot(max(as.Date(d_trval_i)) < min(test_df$dlycaldt))
  run_context[[run_id]] <- list(
    d_trval = d_trval_i,
    X_trval = X_trval_i,
    y_trval = y_trval_i,
    best_params_lgb = list(),
    best_params_xgb = list(),
    best_params_rf = list()
  )
  cat(sprintf("Run %s X_trval shape : %s  (train=%s + val=%s)\n",
              run_id,
              paste(nrow(X_trval_i), ncol(X_trval_i), sep = ", "),
              format(nrow(split_i$X_train), big.mark = ","),
              format(nrow(split_i$X_val),   big.mark = ",")))
}
```

```
## Run A X_trval shape : 393875, 51  (train=349,557 + val=44,318)
## Run B X_trval shape : 393875, 69  (train=349,557 + val=44,318)
```

```
cat("Tuning utilities ready (walk-forward CV by `dlycaldt`) for all runs.\n")
```

```
## Tuning utilities ready (walk-forward CV by `dlycaldt`) for all runs.
```

```
# ── Tuning Cell B: LightGBM (time-series CV) ──────────────────────────────
lgb_param_grid <- expand.grid(
  num_leaves        = c(63),   # c(63, 127, 255),
  min_child_samples = c(20),   # c(20, 100),
  learning_rate     = c(0.05), # c(0.01, 0.05),
  stringsAsFactors = FALSE
)

for (run_id in RUN_LABELS) {
  ctx <- run_context[[run_id]]
  folds_lgb <- make_expanding_date_folds(ctx$d_trval, TS_CV_N_FOLDS, TS_CV_MIN_TRAIN_ROWS)
  if (!length(folds_lgb)) stop(sprintf("No valid time-series folds for LightGBM (Run %s).", run_
id))
  summarize_ts_folds(ctx$d_trval, folds_lgb, sprintf("LightGBM Run %s", run_id))

  cat(sprintf("\n── Run %s LightGBM: %d hyperparameter combos × %d folds %s\n",
              run_id, nrow(lgb_param_grid), length(folds_lgb), strrep("─", 8L)))

  best_mean <- Inf
  best_row <- NULL
  cat_feats <- intersect(c("ticker_enc", "siccd_enc", "naics_enc"), colnames(ctx$X_trval))
  for (j in seq_len(nrow(lgb_param_grid))) {
    params_i <- as.list(lgb_param_grid[j, , drop = FALSE])
    fold_rmses <- numeric(length(folds_lgb))
    for (fi in seq_along(folds_lgb)) {
      tr <- folds_lgb[[fi]]$train
      va <- folds_lgb[[fi]]$val
      dtrain_i <- lgb.Dataset(
        data = data.matrix(ctx$X_trval[tr, , drop = FALSE]),
        label = ctx$y_trval[tr],
        categorical_feature = cat_feats
      )
      dval_i <- lgb.Dataset(
        data = data.matrix(ctx$X_trval[va, , drop = FALSE]),
        label = ctx$y_trval[va],
        reference = dtrain_i
      )
      params_full <- list(
        objective = "regression",
        learning_rate = params_i$learning_rate,
        num_leaves = params_i$num_leaves,
        min_data_in_leaf = params_i$min_child_samples,
        feature_fraction = 0.8,
        bagging_fraction = 0.8,
        verbose = -1
      )
      model_i <- lgb.train(
        params = params_full,
        data = dtrain_i,
        nrounds = 1500,
        valids = list(valid = dval_i),
        early_stopping_rounds = 100,
```

```
        eval = "l2"
      )
      preds <- predict(model_i, data.matrix(ctx$X_trval[va, , drop = FALSE]))
      fold_rmses[fi] <- sqrt(mean((ctx$y_trval[va] - preds)^2))
    }
    mean_rmse <- mean(fold_rmses)
    cat(sprintf("  [Run %s %d/%d] mean_val_RMSE=%f  params=%s\n",
                run_id, j, nrow(lgb_param_grid), mean_rmse,
                paste(names(params_i), unlist(params_i), sep = "=", collapse = ", ")))
    if (mean_rmse < best_mean) {
      best_mean <- mean_rmse
      best_row <- params_i
    }
  }

  run_context[[run_id]]$best_params_lgb <- best_row
  if (length(run_context[[run_id]]$best_params_lgb) == 0L) {
    stop(sprintf("LightGBM tuning did not produce best_params_lgb for Run %s.", run_id))
  }
  cat(sprintf("\nRun %s best params (LightGBM): %s\n",
              run_id,
              paste(names(run_context[[run_id]]$best_params_lgb),
                    unlist(run_context[[run_id]]$best_params_lgb),
                    sep = "=", collapse = ", ")))
  cat(sprintf("Run %s best mean CV RMSE (across folds): %f\n", run_id, best_mean))
}
```

```
##
## ── LightGBM Run A: time-series folds (expanding by date) ──────────────
##   Fold 1: train 78,160 rows  (2014-01-02 → 2015-10-20)  |  val 77,844 rows  (2015-10-21 → 201
7-08-08)
##   Fold 2: train 156,004 rows  (2014-01-02 → 2017-08-08)  |  val 78,656 rows  (2017-08-09 → 20
19-05-29)
##   Fold 3: train 234,660 rows  (2014-01-02 → 2019-05-29)  |  val 79,484 rows  (2019-05-30 → 20
21-03-16)
##   Fold 4: train 314,144 rows  (2014-01-02 → 2021-03-16)  |  val 79,731 rows  (2021-03-17 → 20
22-12-30)
##
## ── Run A LightGBM: 1 hyperparameter combos × 4 folds ────────
##   [Run A 1/1] mean_val_RMSE=0.019016  params=num_leaves=63, min_child_samples=20, learning_ra
te=0.05
##
## Run A best params (LightGBM): num_leaves=63, min_child_samples=20, learning_rate=0.05
## Run A best mean CV RMSE (across folds): 0.019016
##
## ── LightGBM Run B: time-series folds (expanding by date) ──────────────
##   Fold 1: train 78,160 rows  (2014-01-02 → 2015-10-20)  |  val 77,844 rows  (2015-10-21 → 201
7-08-08)
##   Fold 2: train 156,004 rows  (2014-01-02 → 2017-08-08)  |  val 78,656 rows  (2017-08-09 → 20
19-05-29)
##   Fold 3: train 234,660 rows  (2014-01-02 → 2019-05-29)  |  val 79,484 rows  (2019-05-30 → 20
21-03-16)
##   Fold 4: train 314,144 rows  (2014-01-02 → 2021-03-16)  |  val 79,731 rows  (2021-03-17 → 20
22-12-30)
##
## ── Run B LightGBM: 1 hyperparameter combos × 4 folds ────────
##   [Run B 1/1] mean_val_RMSE=0.019017  params=num_leaves=63, min_child_samples=20, learning_ra
te=0.05
##
## Run B best params (LightGBM): num_leaves=63, min_child_samples=20, learning_rate=0.05
## Run B best mean CV RMSE (across folds): 0.019017
```

```
# ── Tuning Cell C: XGBoost (time-series CV) ──────────────────────────────
xgb_param_grid <- expand.grid(
  max_depth        = c(4, 6, 8),
  min_child_weight = c(1, 5),
  learning_rate    = c(0.01, 0.05),
  stringsAsFactors = FALSE
)

for (run_id in RUN_LABELS) {
  ctx <- run_context[[run_id]]
  cat_feats <- intersect(c("ticker_enc", "siccd_enc", "naics_enc"), colnames(ctx$X_trval))

  X_trval_xgb <- ctx$X_trval
  for (col in cat_feats) {
    X_trval_xgb[[col]] <- as.numeric(X_trval_xgb[[col]])
  }

  folds_xgb <- make_expanding_date_folds(ctx$d_trval, TS_CV_N_FOLDS, TS_CV_MIN_TRAIN_ROWS)
  if (!length(folds_xgb)) stop(sprintf("No valid time-series folds for XGBoost (Run %s).", run_i
d))
  summarize_ts_folds(ctx$d_trval, folds_xgb, sprintf("XGBoost Run %s", run_id))

  cat(sprintf("\n── Run %s XGBoost: %d hyperparameter combos × %d folds %s\n",
              run_id, nrow(xgb_param_grid), length(folds_xgb), strrep("─", 8L)))

  best_mean_x <- Inf
  best_row_x <- NULL
  for (j in seq_len(nrow(xgb_param_grid))) {
    params_i <- as.list(xgb_param_grid[j, , drop = FALSE])
    fold_rmses <- numeric(length(folds_xgb))
    for (fi in seq_along(folds_xgb)) {
      tr <- folds_xgb[[fi]]$train
      va <- folds_xgb[[fi]]$val
      dtrain_i <- xgb.DMatrix(
        data = data.matrix(X_trval_xgb[tr, , drop = FALSE]),
        label = ctx$y_trval[tr]
      )
      dval_i <- xgb.DMatrix(
        data = data.matrix(X_trval_xgb[va, , drop = FALSE]),
        label = ctx$y_trval[va]
      )
      params_full <- list(
        objective = "reg:squarederror",
        tree_method = xgb_tree_method,
        eta = params_i$learning_rate,
        max_depth = params_i$max_depth,
        min_child_weight = params_i$min_child_weight,
        subsample = 0.8,
        colsample_bytree = 0.8
      )
      model_i <- xgb.train(
        params = params_full,
```

```
        data = dtrain_i,
        nrounds = 15000,
        watchlist = list(val = dval_i),
        early_stopping_rounds = 100,
        verbose = 0
      )
      preds <- predict(model_i, dval_i)
      fold_rmses[fi] <- sqrt(mean((ctx$y_trval[va] - preds)^2))
    }
    mean_rmse <- mean(fold_rmses)
    cat(sprintf("  [Run %s %d/%d] mean_val_RMSE=%f  params=%s\n",
                run_id, j, nrow(xgb_param_grid), mean_rmse,
                paste(names(params_i), unlist(params_i), sep = "=", collapse = ", ")))
    if (mean_rmse < best_mean_x) {
      best_mean_x <- mean_rmse
      best_row_x <- params_i
    }
  }

  run_context[[run_id]]$best_params_xgb <- best_row_x
  if (length(run_context[[run_id]]$best_params_xgb) == 0L) {
    stop(sprintf("XGBoost tuning did not produce best_params_xgb for Run %s.", run_id))
  }
  cat(sprintf("\nRun %s best params (XGBoost): %s\n",
              run_id,
              paste(names(run_context[[run_id]]$best_params_xgb),
                    unlist(run_context[[run_id]]$best_params_xgb),
                    sep = "=", collapse = ", ")))
  cat(sprintf("Run %s best mean CV RMSE (across folds): %f\n", run_id, best_mean_x))
}
```

```
##
## ── XGBoost Run A: time-series folds (expanding by date) ─────────────
##   Fold 1: train 78,160 rows  (2014-01-02 → 2015-10-20)  |  val 77,844 rows  (2015-10-21 → 201
7-08-08)
##   Fold 2: train 156,004 rows  (2014-01-02 → 2017-08-08)  |  val 78,656 rows  (2017-08-09 → 20
19-05-29)
##   Fold 3: train 234,660 rows  (2014-01-02 → 2019-05-29)  |  val 79,484 rows  (2019-05-30 → 20
21-03-16)
##   Fold 4: train 314,144 rows  (2014-01-02 → 2021-03-16)  |  val 79,731 rows  (2021-03-17 → 20
22-12-30)
##
## ── Run A XGBoost: 12 hyperparameter combos × 4 folds ────────
```

```
##   [Run A 1/12] mean_val_RMSE=0.019018  params=max_depth=4, min_child_weight=1, learning_rate=
0.01
```

```
##   [Run A 2/12] mean_val_RMSE=0.019014  params=max_depth=6, min_child_weight=1, learning_rate=
0.01
```

```
##   [Run A 3/12] mean_val_RMSE=0.019015  params=max_depth=8, min_child_weight=1, learning_rate=
0.01
```

```
##   [Run A 4/12] mean_val_RMSE=0.019017  params=max_depth=4, min_child_weight=5, learning_rate=
0.01
```

```
##   [Run A 5/12] mean_val_RMSE=0.019017  params=max_depth=6, min_child_weight=5, learning_rate=
0.01
```

```
##   [Run A 6/12] mean_val_RMSE=0.019015  params=max_depth=8, min_child_weight=5, learning_rate=
0.01
```

```
##   [Run A 7/12] mean_val_RMSE=0.019015  params=max_depth=4, min_child_weight=1, learning_rate=
0.05
```

```
##   [Run A 8/12] mean_val_RMSE=0.019011  params=max_depth=6, min_child_weight=1, learning_rate=
0.05
```

```
##   [Run A 9/12] mean_val_RMSE=0.019021  params=max_depth=8, min_child_weight=1, learning_rate=
0.05
```

```
##   [Run A 10/12] mean_val_RMSE=0.019018  params=max_depth=4, min_child_weight=5, learning_rate
=0.05
```

```
##   [Run A 11/12] mean_val_RMSE=0.019011  params=max_depth=6, min_child_weight=5, learning_rate
=0.05
```

```
##   [Run A 12/12] mean_val_RMSE=0.019017  params=max_depth=8, min_child_weight=5, learning_rate
=0.05
##
## Run A best params (XGBoost): max_depth=6, min_child_weight=1, learning_rate=0.05
## Run A best mean CV RMSE (across folds): 0.019011
##
## ── XGBoost Run B: time-series folds (expanding by date) ──────────
##   Fold 1: train 78,160 rows  (2014-01-02 → 2015-10-20)  |  val 77,844 rows  (2015-10-21 → 201
7-08-08)
##   Fold 2: train 156,004 rows  (2014-01-02 → 2017-08-08)  |  val 78,656 rows  (2017-08-09 → 20
19-05-29)
##   Fold 3: train 234,660 rows  (2014-01-02 → 2019-05-29)  |  val 79,484 rows  (2019-05-30 → 20
21-03-16)
##   Fold 4: train 314,144 rows  (2014-01-02 → 2021-03-16)  |  val 79,731 rows  (2021-03-17 → 20
22-12-30)
##
## ── Run B XGBoost: 12 hyperparameter combos × 4 folds ─────────
```

```
##   [Run B 1/12] mean_val_RMSE=0.019018  params=max_depth=4, min_child_weight=1, learning_rate=
0.01
```

```
##   [Run B 2/12] mean_val_RMSE=0.019018  params=max_depth=6, min_child_weight=1, learning_rate=
0.01
```

```
##   [Run B 3/12] mean_val_RMSE=0.019017  params=max_depth=8, min_child_weight=1, learning_rate=
0.01
```

```
##   [Run B 4/12] mean_val_RMSE=0.019018  params=max_depth=4, min_child_weight=5, learning_rate=
0.01
```

```
##   [Run B 5/12] mean_val_RMSE=0.019017  params=max_depth=6, min_child_weight=5, learning_rate=
0.01
```

```
##   [Run B 6/12] mean_val_RMSE=0.019019  params=max_depth=8, min_child_weight=5, learning_rate=
0.01
```

```
##   [Run B 7/12] mean_val_RMSE=0.019016  params=max_depth=4, min_child_weight=1, learning_rate=
0.05
```

```
##   [Run B 8/12] mean_val_RMSE=0.019022  params=max_depth=6, min_child_weight=1, learning_rate=
0.05
```

```
##   [Run B 9/12] mean_val_RMSE=0.019011  params=max_depth=8, min_child_weight=1, learning_rate=
0.05
```

```
##   [Run B 10/12] mean_val_RMSE=0.019018  params=max_depth=4, min_child_weight=5, learning_rate
=0.05
```

```
##   [Run B 11/12] mean_val_RMSE=0.019020  params=max_depth=6, min_child_weight=5, learning_rate
=0.05
```

```
##   [Run B 12/12] mean_val_RMSE=0.019018  params=max_depth=8, min_child_weight=5, learning_rate
=0.05
##
## Run B best params (XGBoost): max_depth=8, min_child_weight=1, learning_rate=0.05
## Run B best mean CV RMSE (across folds): 0.019011
```

```
# ── Tuning Cell E: Refit on train+val → evaluate on test ──────────────────
for (run_id in RUN_LABELS) {
  split_i <- RUN_SPLITS[[run_id]]
  ctx <- run_context[[run_id]]
  cat_feats <- intersect(c("ticker_enc", "siccd_enc", "naics_enc"), colnames(ctx$X_trval))

  # ── 1. LightGBM tuned refit ──────────────────────────────────────────────
  dtrval_lgb <- lgb.Dataset(
    data = data.matrix(ctx$X_trval),
    label = ctx$y_trval,
    categorical_feature = cat_feats
  )
  lgb_tuned_params <- list(
    objective = "regression",
    learning_rate    = ctx$best_params_lgb$learning_rate,
    num_leaves       = ctx$best_params_lgb$num_leaves,
    min_data_in_leaf = ctx$best_params_lgb$min_child_samples,
    feature_fraction = 0.8,
    bagging_fraction = 0.8,
    verbose = -1
  )
  lgb_tuned <- lgb.train(
    params = lgb_tuned_params,
    data = dtrval_lgb,
    nrounds = 1500
  )
  lgb_tuned_preds <- predict(lgb_tuned, data.matrix(split_i$X_test))
  results[[paste(run_id, "LightGBM_tuned", sep = "::")]] <- evaluate(
    split_i$y_test,
    lgb_tuned_preds,
    label = sprintf("Run %s (%s) LightGBM tuned — Test set (2023-2024)",
                    run_id,
                    if (exists("RUN_NAMES") && run_id %in% names(RUN_NAMES)) RUN_NAMES[[run_id]]
else run_id)
  )

  # ── 2. XGBoost tuned refit ──────────────────────────────────────────────
  X_trval_xgb <- ctx$X_trval
  X_test_xgb <- split_i$X_test
  for (col in cat_feats) {
    X_trval_xgb[[col]] <- as.numeric(X_trval_xgb[[col]])
    X_test_xgb[[col]] <- as.numeric(X_test_xgb[[col]])
  }
  dtrval_xgb <- xgb.DMatrix(data = data.matrix(X_trval_xgb), label = ctx$y_trval)
  dtest_xgb_refit <- xgb.DMatrix(data = data.matrix(X_test_xgb), label = split_i$y_test)
  xgb_tuned_params <- list(
    objective = "reg:squarederror",
    tree_method = xgb_tree_method,
    eta = ctx$best_params_xgb$learning_rate,
    max_depth = ctx$best_params_xgb$max_depth,
    min_child_weight = ctx$best_params_xgb$min_child_weight,
    subsample = 0.8,
```

```
    colsample_bytree = 0.8
  )
  xgb_tuned <- xgb.train(
    params = xgb_tuned_params,
    data = dtrval_xgb,
    nrounds = 1500,
    verbose = 0
  )
  xgb_tuned_preds <- predict(xgb_tuned, dtest_xgb_refit)
  results[[paste(run_id, "XGBoost_tuned", sep = "::")]] <- evaluate(
    split_i$y_test,
    xgb_tuned_preds,
    label = sprintf("Run %s (%s) XGBoost tuned — Test set (2023-2024)",
                    run_id,
                    if (exists("RUN_NAMES") && run_id %in% names(RUN_NAMES)) RUN_NAMES[[run_id]]
else run_id)
  )

  if (is.null(models_by_run[[run_id]])) models_by_run[[run_id]] <- list()
  models_by_run[[run_id]]$lgb_tuned <- lgb_tuned
  models_by_run[[run_id]]$xgb_tuned <- xgb_tuned
}
```

```
##
## ── Run A (Strict no-transcript) LightGBM tuned — Test set (2023-2024)
##   RMSE                    : 0.016641
##   MAE                     : 0.011461
##   R2                      : -0.028235
##   Directional_Accuracy    : 0.501633
##   Spearman_IC             : 0.015593
##
## ── Run A (Strict no-transcript) XGBoost tuned — Test set (2023-2024)
##   RMSE                    : 0.016621
##   MAE                     : 0.011433
##   R2                      : -0.025777
##   Directional_Accuracy    : 0.504074
##   Spearman_IC             : 0.019152
##
## ── Run B (Full transcript) LightGBM tuned — Test set (2023-2024)
##   RMSE                    : 0.016720
##   MAE                     : 0.011547
##   R2                      : -0.038048
##   Directional_Accuracy    : 0.496830
##   Spearman_IC             : 0.008879
##
## ── Run B (Full transcript) XGBoost tuned — Test set (2023-2024)
##   RMSE                    : 0.016859
##   MAE                     : 0.011677
##   R2                      : -0.055326
##   Directional_Accuracy    : 0.501430
##   Spearman_IC             : 0.010817
```

```
# ── Summary comparison ──────────────────────────────────────────────
cat("\n\n== Model comparison: baseline vs tuned ==\n")
```

```
##
##
## == Model comparison: baseline vs tuned ==
```

```
compare_df <- dplyr::bind_rows(results, .id = "model")
model_parts <- do.call(rbind, strsplit(compare_df$model, "::", fixed = TRUE))
compare_df$run <- model_parts[, 1]
compare_df$model_name <- model_parts[, 2]
compare_df$run_name <- ifelse(compare_df$run %in% names(RUN_NAMES), RUN_NAMES[compare_df$run], c
ompare_df$run)
compare_df <- compare_df[, c("run", "run_name", "model_name", "RMSE", "MAE", "R2", "Directional_
Accuracy", "Spearman_IC")]
compare_df <- compare_df[order(compare_df$run, compare_df$model_name), ]
print(tibble::as_tibble(compare_df))
```

```
## # A tibble: 8 × 8
##   run   run_name          model_name   RMSE    MAE       R2 Directional_Accuracy
##   <chr> <chr>             <chr>       <dbl>  <dbl>    <dbl>                <dbl>
## 1 A     Strict no-transc… LightGBM   0.0164 0.0112  3.54e-4                0.521
## 2 A     Strict no-transc… LightGBM_… 0.0166 0.0115 -2.82e-2                0.502
## 3 A     Strict no-transc… XGBoost    0.0164 0.0112  3.90e-4                0.520
## 4 A     Strict no-transc… XGBoost_t… 0.0166 0.0114 -2.58e-2                0.504
## 5 B     Full transcript   LightGBM   0.0164 0.0112  4.06e-4                0.522
## 6 B     Full transcript   LightGBM_… 0.0167 0.0115 -3.80e-2                0.497
## 7 B     Full transcript   XGBoost    0.0164 0.0112  7.98e-4                0.519
## 8 B     Full transcript   XGBoost_t… 0.0169 0.0117 -5.53e-2                0.501
## # ℹ 1 more variable: Spearman_IC <dbl>
```

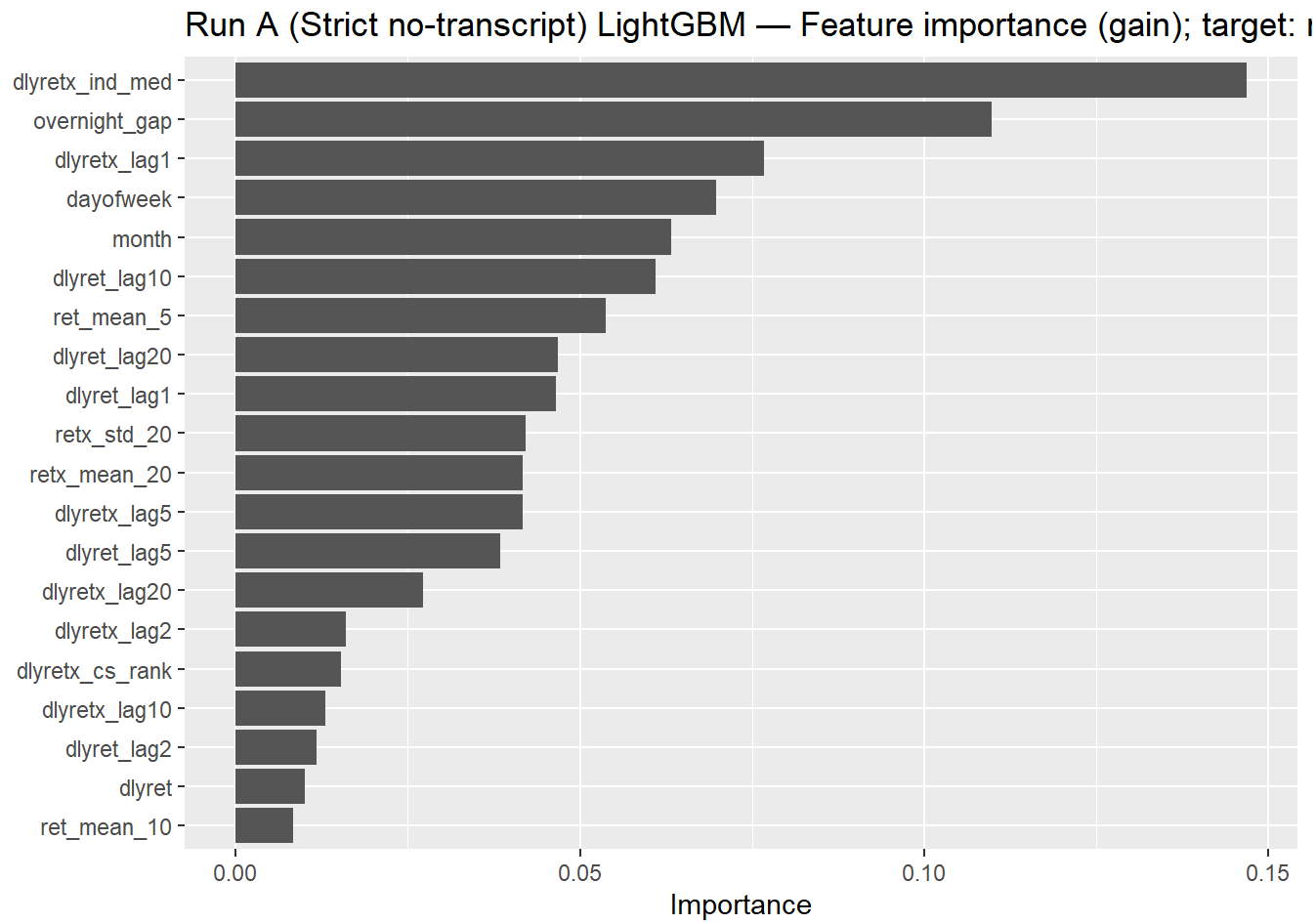
```
# Strict incremental comparison: Run B - Run A by model_name
metric_cols <- c("RMSE", "MAE", "R2", "Directional_Accuracy", "Spearman_IC")
compare_a <- compare_df[compare_df$run == "A", c("model_name", metric_cols), drop = FALSE]
compare_b <- compare_df[compare_df$run == "B", c("model_name", metric_cols), drop = FALSE]
delta_df <- merge(compare_b, compare_a, by = "model_name", suffixes = c("_B", "_A"), all = FALS
E)
for (m in metric_cols) {
  delta_df[[paste0("Delta_", m, "_B_minus_A")]] <- delta_df[[paste0(m, "_B")]] - delta_df[[paste
0(m, "_A")]]
}
cat("\n\n== Incremental effect: Run B - Run A ==\n")
```

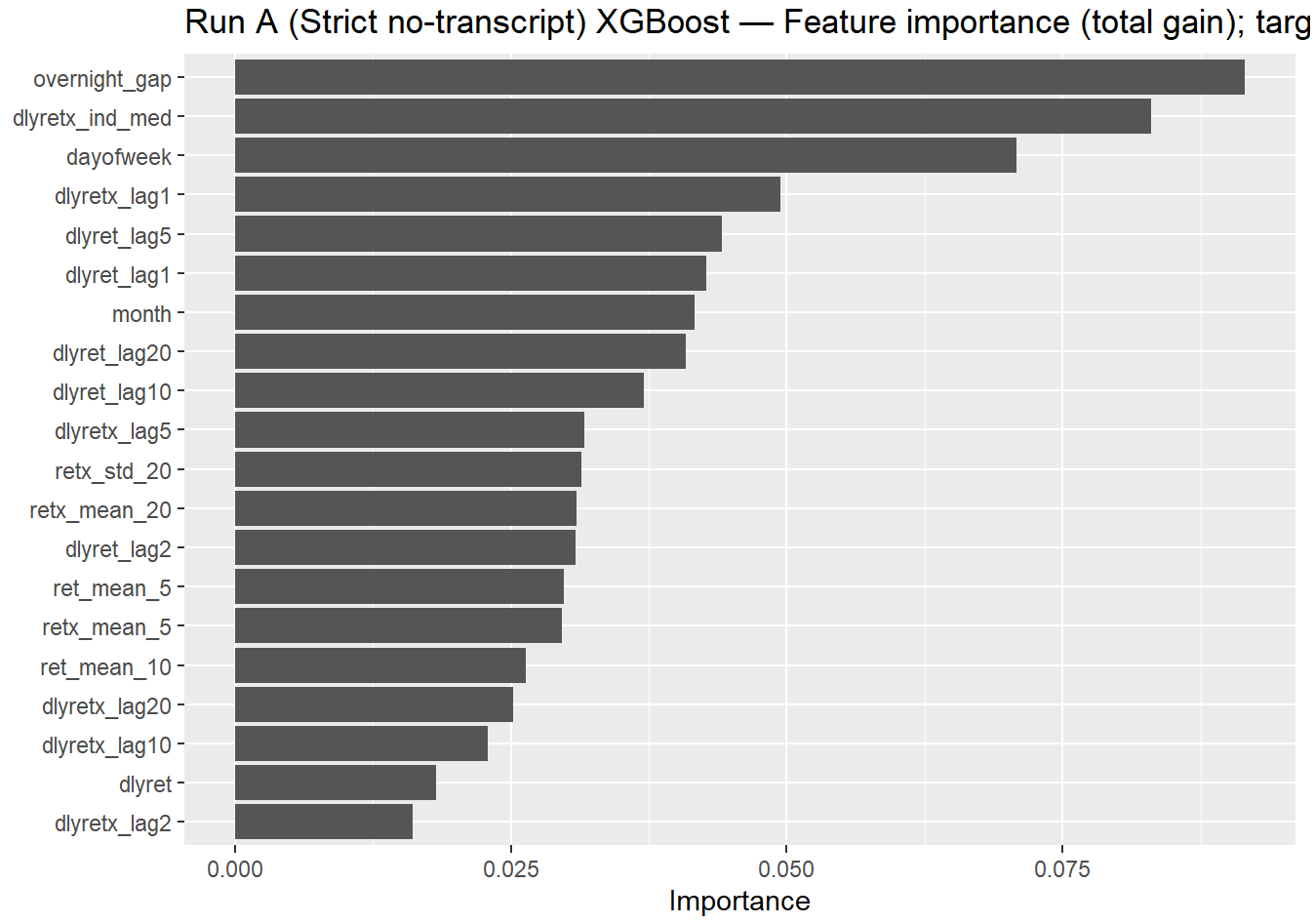
```
##
##
## == Incremental effect: Run B - Run A ==
```

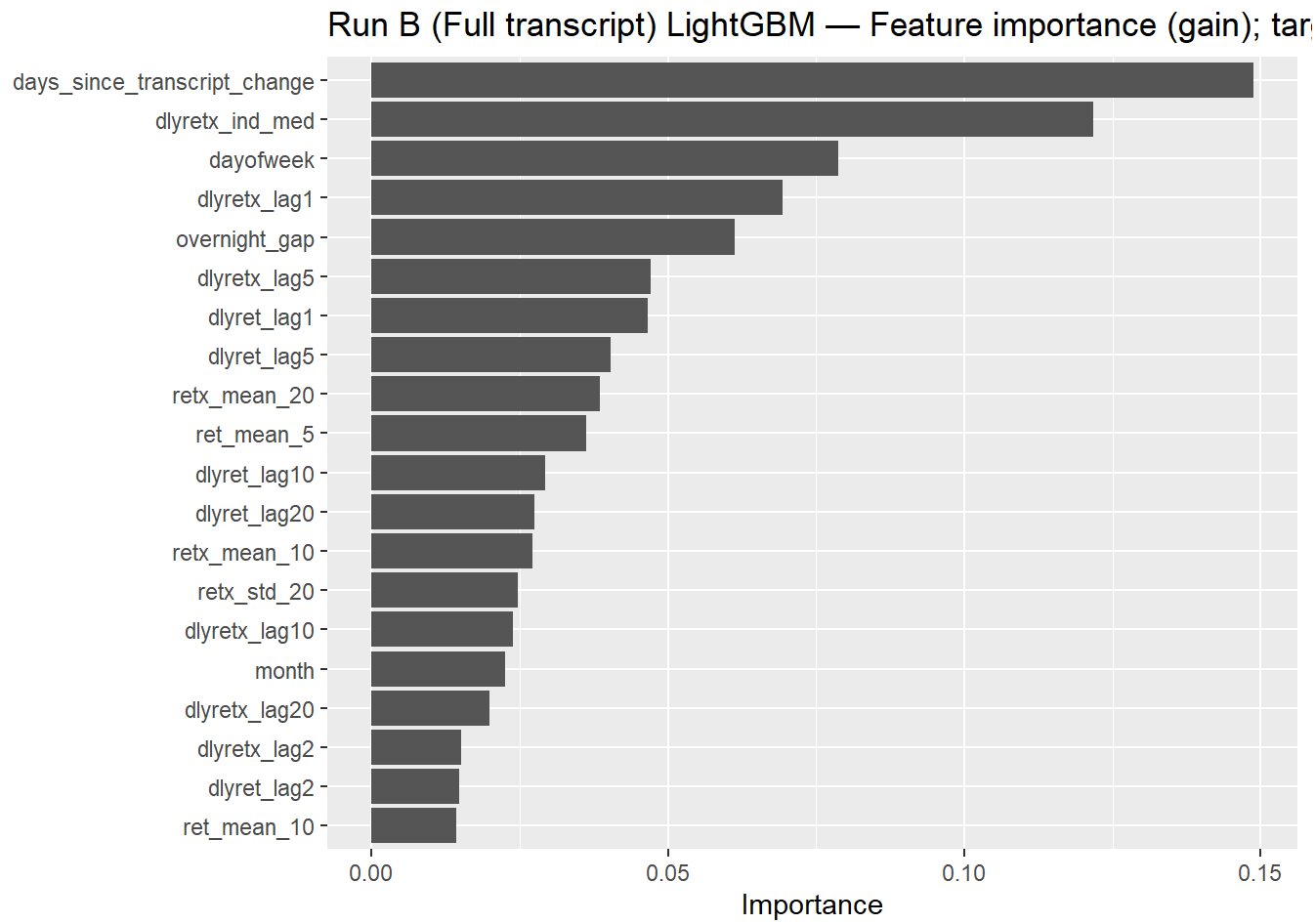
```
print(tibble::as_tibble(delta_df))
```

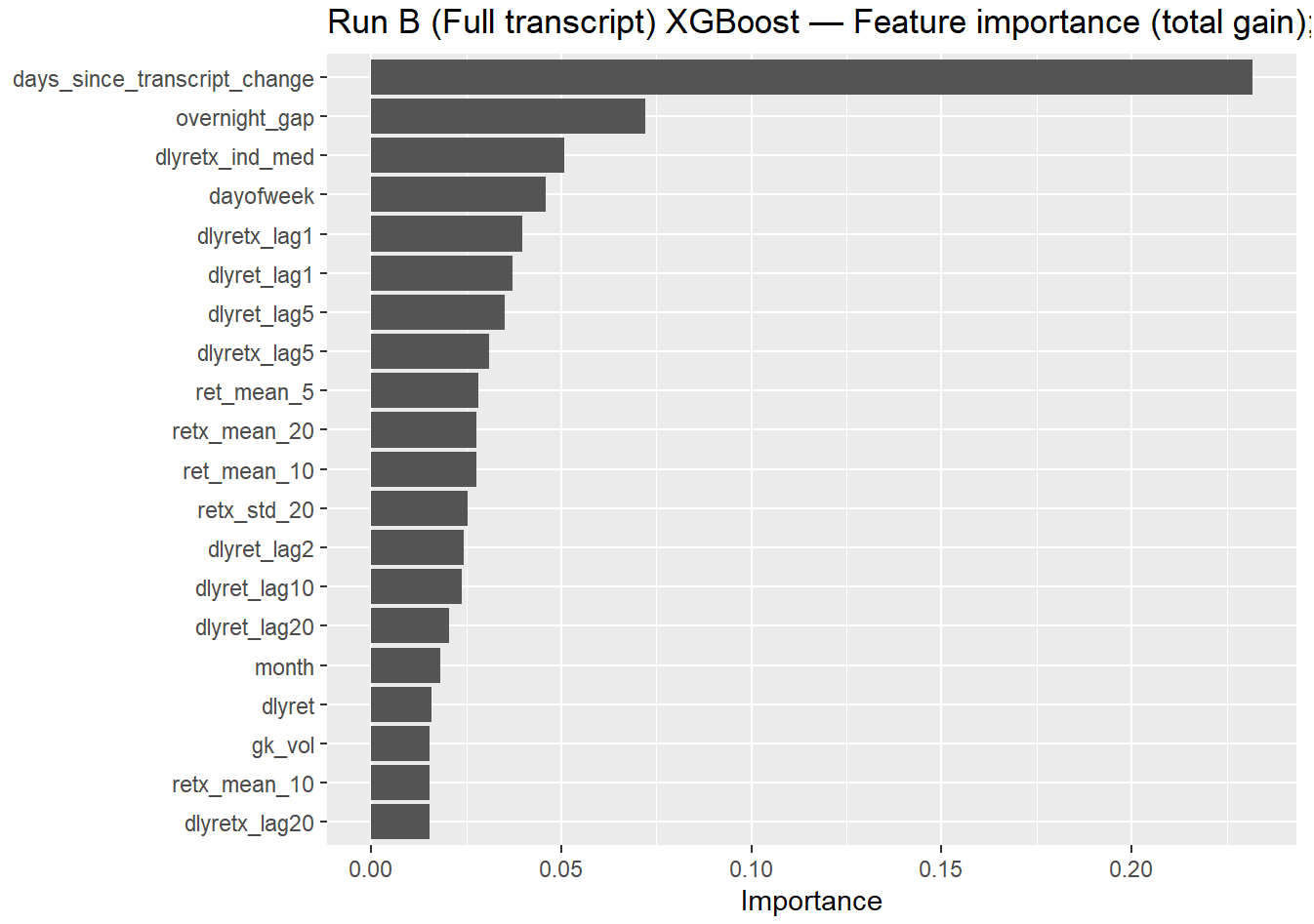
```
## # A tibble: 4 × 16
##   model_name  RMSE_B  MAE_B     R2_B Directional_Accuracy_B Spearman_IC_B RMSE_A
##   <chr>        <dbl>  <dbl>    <dbl>                  <dbl>         <dbl>  <dbl>
## 1 LightGBM    0.0164 0.0112  4.06e-4                  0.522      -0.00406 0.0164
## 2 LightGBM_t… 0.0167 0.0115 -3.80e-2                  0.497       0.00888 0.0166
## 3 XGBoost     0.0164 0.0112  7.98e-4                  0.519       0.00826 0.0164
## 4 XGBoost_tu… 0.0169 0.0117 -5.53e-2                  0.501       0.0108  0.0166
## # ℹ 9 more variables: MAE_A <dbl>, R2_A <dbl>, Directional_Accuracy_A <dbl>,
## #   Spearman_IC_A <dbl>, Delta_RMSE_B_minus_A <dbl>, Delta_MAE_B_minus_A <dbl>,
## #   Delta_R2_B_minus_A <dbl>, Delta_Directional_Accuracy_B_minus_A <dbl>,
## #   Delta_Spearman_IC_B_minus_A <dbl>
```

```
# ── Step 5: Feature importance (gain) — baseline LightGBM / XGBoost ──────────
plot_importance_bars <- function(importances, feature_names, title, top_n = 20L) {
  importances <- as.numeric(importances)
  feature_names <- as.character(feature_names)
  ord <- order(importances)
  k <- min(top_n, length(importances))
  idx <- tail(ord, k)
  df_imp <- data.frame(
    feature = feature_names[idx],
    importance = importances[idx],
    stringsAsFactors = FALSE
  )
  ggplot(df_imp, aes(x = .data$importance, y = reorder(.data$feature, .data$importance))) +
    geom_col() +
    labs(title = title, x = "Importance", y = NULL)
}

for (run_id in RUN_LABELS) {
  run_name <- if (run_id %in% names(RUN_NAMES)) RUN_NAMES[[run_id]] else run_id
  if (!is.null(models_by_run[[run_id]]$lgb_model)) {
    lgb_imp_df <- lightgbm::lgb.importance(models_by_run[[run_id]]$lgb_model)
    print(plot_importance_bars(
      lgb_imp_df$Gain, lgb_imp_df$Feature,
      sprintf("Run %s (%s) LightGBM — Feature importance (gain); target: %s", run_id, run_name,
TARGET_COL)
    ))
  }

  if (!is.null(models_by_run[[run_id]]$xgb_model)) {
    xgb_imp_df <- xgboost::xgb.importance(model = models_by_run[[run_id]]$xgb_model)
    print(plot_importance_bars(
      xgb_imp_df$Gain, xgb_imp_df$Feature,
      sprintf("Run %s (%s) XGBoost — Feature importance (total gain); target: %s", run_id, run_n
ame, TARGET_COL)
    ))
  }
}
```


Run A (Strict no-transcript) LightGBM — Feature importance (gain); target: r
dlyretx_ind_med
overnight_gap
dlyretx_lag1
dayofweek
month
dlyret_lag10
ret_mean_5
dlyret_lag20
dlyret_lag1
retx_std_20
retx_mean_20
dlyretx_lag5
dlyret_lag5
dlyretx_lag20
dlyretx_lag2
dlyretx_cs_rank
dlyretx_lag10
dlyret_lag2
dlyret
ret_mean_10
0.00
0.05
0.10
0.15
Importance


Run A (Strict no-transcript) XGBoost — Feature importance (total gain); targ
overnight_gap
dlyretx_ind_med
dayofweek
dlyretx_lag1
dlyret_lag5
dlyret_lag1
month
dlyret_lag20
dlyret_lag10
dlyretx_lag5
retx_std_20
retx_mean_20
dlyret_lag2
ret_mean_5
retx_mean_5
ret_mean_10
dlyretx_lag20
dlyretx_lag10
dlyret
dlyretx_lag2
0.000
0.025
0.050
0.075
Importance


Run B (Full transcript) LightGBM — Feature importance (gain); tar
days_since_transcript_change
dlyretx_ind_med
dayofweek
dlyretx_lag1
overnight_gap
dlyretx_lag5
dlyret_lag1
dlyret_lag5
retx_mean_20
ret_mean_5
dlyret_lag10
dlyret_lag20
retx_mean_10
retx_std_20
dlyretx_lag10
month
dlyretx_lag20
dlyretx_lag2
dlyret_lag2
ret_mean_10
0.00
0.05
0.10
0.15
Importance


Run B (Full transcript) XGBoost — Feature importance (total gain)
days_since_transcript_change
overnight_gap
dlyretx_ind_med
dayofweek
dlyretx_lag1
dlyret_lag1
dlyret_lag5
dlyretx_lag5
ret_mean_5
retx_mean_20
ret_mean_10
retx_std_20
dlyret_lag2
dlyret_lag10
dlyret_lag20
month
dlyret
gk_vol
retx_mean_10
dlyretx_lag20
0.00
0.05
0.10
0.15
0.20
Importance

```
results_ic <- dplyr::bind_rows(results, .id = "model")
parts <- do.call(rbind, strsplit(results_ic$model, "::", fixed = TRUE))
results_ic$run <- parts[, 1]
results_ic$model_name <- parts[, 2]
results_ic <- results_ic[, c("run", "model_name", "RMSE", "MAE", "R2", "Directional_Accuracy",
"Spearman_IC")]
results_ic <- results_ic[order(-results_ic$Spearman_IC), ]
cat("\n== Best model by Spearman IC (test 2023–2024) ==\n")
```

```
##
## == Best model by Spearman IC (test 2023–2024) ==
```

```
cat(sprintf("Run %s | %s  (IC = %f)\n",
            results_ic$run[1], results_ic$model_name[1], results_ic$Spearman_IC[1]))
```

```
## Run A | XGBoost_tuned  (IC = 0.019152)
```